OFFICIAL TECHNICAL SPECIFICATION & COMPREHENSIVE BUILD MANUAL

CloudRover V1.0

Autonomous Tracked Mini Robot • Complete Zero-to-Hero Build Guide

CAD Blueprints • Dual-Charging Power Architecture • Pin-by-Pin Schematics • Production Firmware



CHASSIS & MECHANICS • 80 x 80 x 85 mm Cubic Enclosure • Molded Track Tray with Rubber Treads • Dual Geared DC Motors + Ball Caster	**POWER ARCHITECTURE** • 2x 18650 Li-ion Cells (5200+ mAh) • Monocrystalline Solar Roof (1W) • Single USB-C + LM2596 Regulated 5.00V
BRAINS & CONNECTIVITY • Primary: ESP32 NodeMCU DevKit V1 • Vision: ESP32-CAM (OV2640 MJPEG) • 4G LTE: SIM7600E-H (Voice + SMS)	**SENSING & INTERACTIVE HMI** • VL53L1X Laser Distance Sensor (ToF) • ST7735 1.8-inch TFT Expressive Eyes • MAX9814 AGC Mic & PAM8403 8Ω Audio

Follow the Phases sequentially (1 to 7). Tick every inspection box. Strictly execute the 5.00V calibration in Phase 3.

Academic Defense Release — Production Verified

Table of Contents

Table of Contents 2

Executive Briefing & Operational Guidelines 3

Read this first: how to use this manual, and what is verified . . . 3

Required Workshop Tooling & Equipment Checklist . . . 3

Project Architecture Progression Roadmap . . . 3

PHASE 1 — Project Overview, Defense Value & Shopping Checklist 4

1.1 The idea . . . 4

1.2 Problem solved . . . 4

1.3 Use cases . . . 4

1.4 Master Bill of Materials (tick when bought AND inspected) . . . 6

1.5 The 5-Point Field Guide for buying genuine 18650 batteries . . . 7

1.6 Power configuration decision (choose A or B now) . . . 7

PHASE 2 — Component Dimensions & 3D CAD Modeling / Printing Guide 8

2.1 Component dimension table (L x W x H, mm, typical) . . . 8

2.2 The 4 modular printed parts (summary) . . . 8

2.3 Part 1 — Base Track Frame (Fusion 360 / SolidWorks) . . . 9

2.4 Part 2 — Main Cubic Body Shell 80 x 80 x 85 mm, walls 2.5 mm . . . 10

2.5 Part 3 — Removable Front Faceplate (+0.4 mm tolerance on every cut) . . . 11

2.6 Part 4 — Top Solar Lid . . . 11

2.7 Slicer settings (Cura / PrusaSlicer / Bambu Studio) . . . 12

PHASE 3 — Bench Calibration & Power Distribution Assembly 14

Step 3.1 — Solder the 1N5819 Schottky diode to the solar panel and TP4056 IN+ / IN- . . . 14

Step 3.2 (CRITICAL) — Set and calibrate the regulator to exactly 5.00 V . . . 14

Step 3.3 — Build the 5 x 7 cm perfboard power rail (5 V, 3.3 V, GND) . . . 14

PHASE 4 — Pin-by-Pin Signal & Audio Wiring 16

4.1 Master pinout table (ESP32 NodeMCU DevKit V1) . . . 16

4.2 Sequential wiring checklist . . . 16

4.3 Audio wiring . . . 17

4.4 Pre-power inspection checklist . . . 17

PHASE 5 — Arduino IDE Setup & Ready-to-Upload Source Code 19

Step 5.1 — Install the tools . . . 19

Step 5.2 — Full Code #1: Main ESP32 NodeMCU firmware . . . 19

Step 5.3 — Full Code #2: ESP32-CAM live MJPEG stream . . . 23

PHASE 6 — Final Physical Assembly & Stacking 25

Assembly order . . . 25

PHASE 7 — First Boot Testing, Troubleshooting & Defense Guide 27

7.1 The 6-step first-power-on test protocol . . . 27

7.2 Troubleshooting table . . . 28

7.3 Top 5 questions from the graduation committee — model answers . . . 28

7.4 Five-minute defense presentation script . . . 29

7.5 Final acceptance checklist . . . 29

Executive Briefing & Operational Guidelines

Read this first: how to use this manual, and what is verified

- Every part dimension in this guide is the **typical** value for that part. Cheap modules vary by 1-3 mm between sellers. **Measure each part with calipers when it arrives** and compare with Phase 2 before you press Print. Phase 2 tells you exactly which numbers to re-check.
- The firmware and wiring are written to be consistent with each other (every GPIO is used once; see the pin table in Phase 4). I could not run it on your exact hardware, so the Phase 7 test protocol is how you prove it. Fix the problems in the order the protocol lists them.
- **Honest scope:** voice calls, SMS remote control and the intercom run over 4G. The live **video** stream runs over WiFi (ESP32-CAM has no modem link). Over the internet it needs the phone hotspot or router port-forward described in Phase 5. Do not claim 4G video in your defense; the Q&A in Phase 7 covers this.
- Two power configurations are documented (A: 2S, B: 1S2P). The step lists show both. Pick ONE before you buy parts (Phase 1).

[CRITICAL SAFETY WARNING]

Li-ion safety: never short 18650 cells, never charge a cell below 2.5 V or a swollen cell, never leave charging unattended the first time, keep a bowl of sand or a metal tray nearby. Use only protected charging boards. Solder in ventilated air.

Required Workshop Tooling & Equipment Checklist

Tool / Equipment	Minimum Specification	Purpose in Assembly
Digital Multimeter	DC Volts (0.01V resolution), Continuity beeper	Calibrating 5.00V rail & checking shorts
Soldering Station	Temperature controlled (set to 350°C)	Wiring perfboard, bus rails, and caps
Digital Vernier Caliper	0.05 mm accuracy	Verifying printed parts & module dimensions
Wire Stripper & Cutters	AWG 20-30 precision cutters	Pre-tinning 0.5mm power wires & Dupont looms
Heat Gun / Lighter	For heat-shrink tubing	Insulating diode and power joints
Fine Tweezers & Kapton Tape	Anti-static, heat resistant	Holding wire bundles and securing antenna

Project Architecture Progression Roadmap

Phase	Focus Area	Key Deliverables
Phase 1	Bill of Materials & Shopping Guide	Component procurement, 5-point battery testing, Config A/B selection
Phase 2	3D CAD Modeling & Slicing	CAD models of 4 modular parts, slicer settings, orthographic blueprint
Phase 3	Power Calibration & Bus Assembly	Solar diode wiring, 5.00V trimmer calibration, heavy 0.5mm copper rails
Phase 4	Pin-by-Pin Master Wiring	Complete GPIO logic mapping, dual-mic split, passive audio mixer
Phase 5	Firmware Upload & 4G Config	Dual-core FreeRTOS controller firmware, ESP32-CAM live video server
Phase 6	Mechanical Integration & Stacking	4-tier brass standoff stacking, wire routing, final enclosure fitment
Phase 7	Testing, Troubleshooting & Defense	6-step commissioning tests, troubleshooting matrix, graduation defense Q&A;

PHASE 1 — Project Overview, Defense Value & Shopping Checklist

1.1 The idea

CloudRover is a compact tracked robot in an 80 x 80 x 85 mm cube. It is remote-controllable from anywhere with 4G coverage (SMS commands and two-way voice call), shows a live camera view on WiFi, keeps itself safe with a laser distance sensor, and recharges itself from a solar roof. A 1.8-inch screen shows animated eyes, and a synthesised voice/tone system reacts to the world (startup greeting, obstacle alarm, clap reaction, solar charging chime) **in sync with the eyes**.

1.2 Problem solved

- **Range:** WiFi/Bluetooth robots stop at the router. A SIM7600E-H LTE modem gives control and voice anywhere with a mobile signal.
- **Endurance:** the solar roof trickle-charges the cells outdoors or near a window; it extends run time, it does not replace charging (about 150-250 mA in direct sun).
- **Safety:** an independent laser stop at 150 mm and a 1.5 s dead-man timeout stop the robot if commands or the link are lost.

1.3 Use cases

- **Security patrol / remote check:** call the robot, hear and talk through the intercom, look through the camera on the local network.
- **Telepresence:** a talking, expressive 'face' you can drive to a person in another room.
- **Confined-space inspection:** 80 mm width fits under furniture and in ducts; camera and laser show what is ahead.
- **Education:** demonstrates power electronics, sensor fusion, wireless comms and CAD in one project.

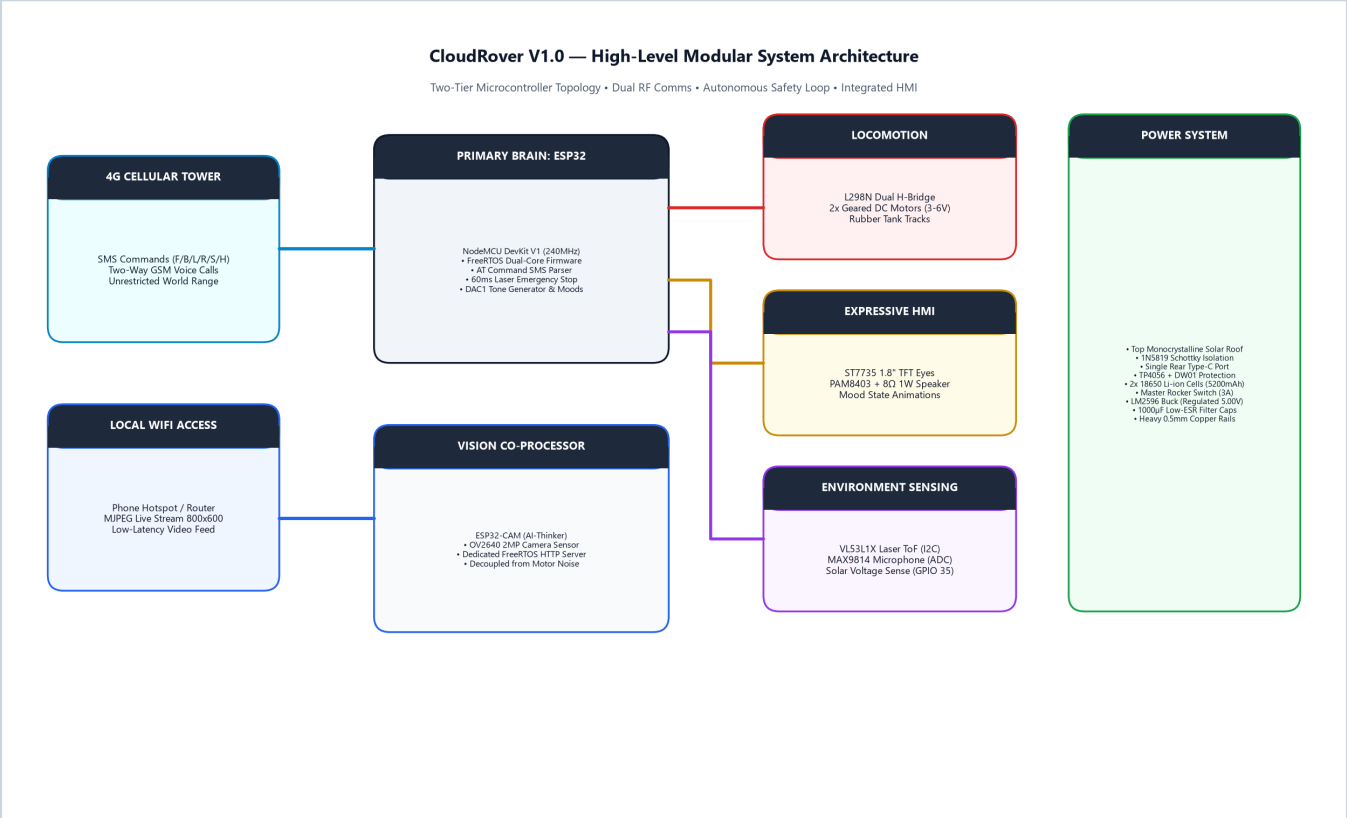


Figure 1.1 — High-Level Modular System Architecture: Two-Tier Microcontroller Topology, 4G Cellular Intercom & Local WiFi Streaming.

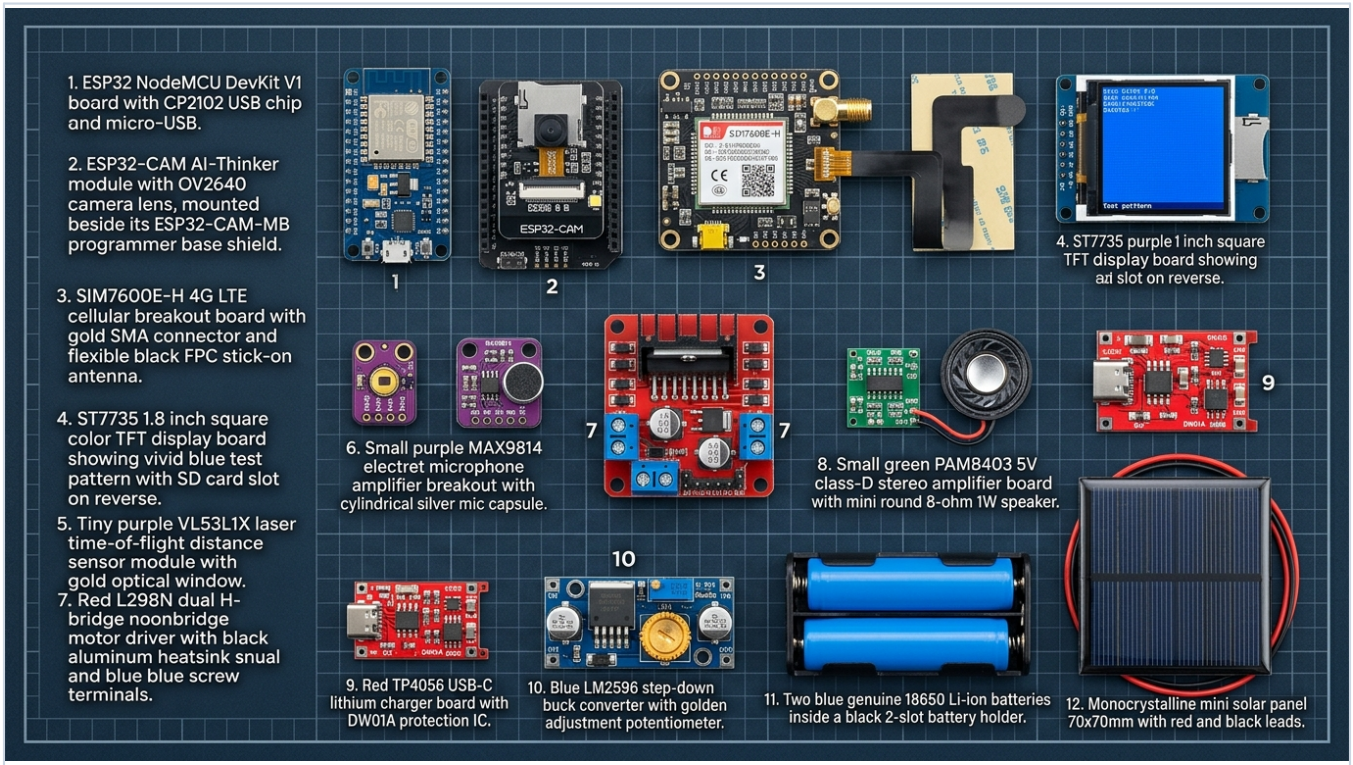


Figure 1.2 — Electronics Modules Catalog: Dual ESP32 Microcontrollers, 4G LTE Cat-4 Modem, Color TFT, Sensors, Drivers & Dual-Charging BMS.

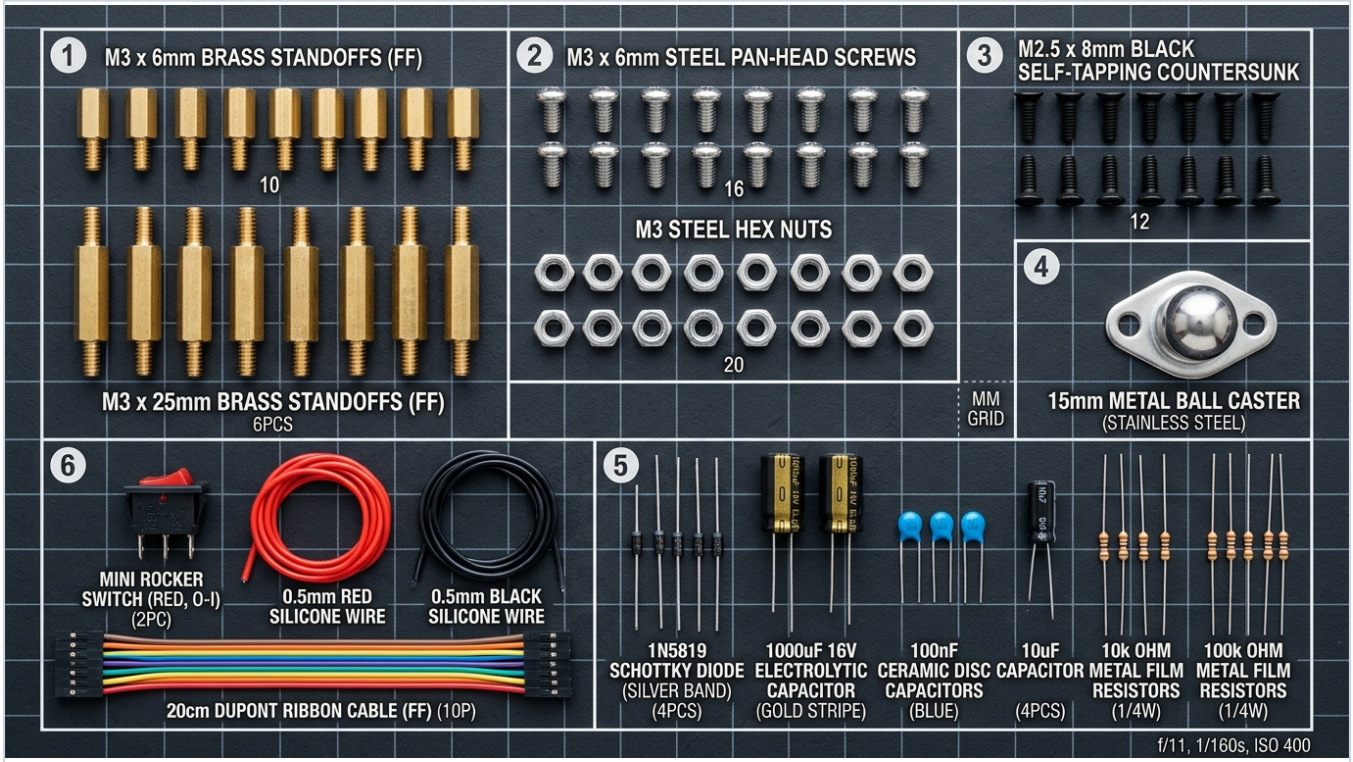


Figure 1.3 — Fasteners, Standoffs, Passives & Wiring Catalog: M3 Hardware, 1N5819 Diodes, Capacitors, Precision Resistors & Connectors.

1.4 Master Bill of Materials (tick when bought AND inspected)

[ ]	Qty	Item	Specification / what to check
[ ]	1	Mini tank tracks chassis kit	2 DC geared motors 3-6 V, tracks, wheels, plate. Measure plate/motor size (Phase 2)
[ ]	1	Rear metal ball caster	Metal, M3 mounting, height matched to track chassis
[ ]	1	ESP32 NodeMCU DevKit V1	ESP32-WROOM-32, 30 or 38 pin; CP2102/CH340 USB; has GPIO 25 and 34 broken out
[ ]	1	ESP32-CAM (OV2640) + MB shield	AI-Thinker, with PSRAM, MB = USB programmer base
[ ]	1	SIM7600E-H 4G module on breakout	E-H = Europe/Middle East/Africa band set; check your carrier bands; UART + MIC/EAR pins exposed; 5 V input
[ ]	1	4G FPC antenna + IPEX-to-SMA adapter	Stick-on FPC antenna goes on the inner wall (Phase 6). Adapter only if module has SMA
[ ]	1	Nano or Micro SIM (match your module)	Data + voice enabled. PIN lock disabled. Test it in a phone first
[ ]	1	1.8" ST7735 TFT 128x160 SPI	8-pin SPI (VCC, GND, CS, RST, DC/A0, SDA, SCK, LED); 3.3 V logic
[ ]	1	VL53L0X (GY-530) or VL53L1X ToF	I2C, 3.3 V-safe VIN. L1X reaches 4 m, L0X about 1.2 m
[ ]	1	MAX9814 microphone amplifier	Auto-gain; AR pin left floating; 3.3 V supply
[ ]	1	L298N motor driver module	Dual H-bridge, 43 x 43 mm. Remove ENA/ENB jumpers
[ ]	1	PAM8403 5 V class-D amp module	With volume pot or plain 3-pin input
[ ]	1	Speaker 8 ohm 1 W mini	20-28 mm round or small rectangular
[ ]	2	Original 18650 Li-ion 2600-3000 mAh	Genuine, same brand/batch (use Field Guide 1.5)
[ ]	1	2-slot 18650 holder with leads	Wires at least 20 cm. Parallel type for Config B
[ ]	1	TP4056 Type-C board with DW01A	Config B. Board has IN+/IN-, B+/B-, OUT+/OUT- pads
[ ]	1	2S Type-C charger/BMS board (8.4 V)	Config A only. Replaces TP4056 (TP4056 cannot charge 2 cells in series)
[ ]	1	MT3608 boost converter module	Config B (1S2P): raises 3.0-4.2 V to 5.00 V. Use a 2 A rated module
[ ]	1	LM2596 buck converter module	Config A (2S): input 7.4 V down to 5.00 V. Input must be above ~6.5 V.
[ ]	1	Mini solar panel 5-6 V (60x60/70x70)	Epoxy-coated flat panel with 2 leads, under 3 mm thick
[ ]	1	1N5819 Schottky diode	Axial, silver stripe = cathode. Blocks night-time back-flow
[ ]	1	Mini rocker switch (snap-in 3 A)	Snap-in, rated 3 A or more. Measure the hole (Phase 2)
[ ]	2	Electrolytic capacitor 1000 uF 16 V	One for SIM7600, one for ESP32-CAM supply (plus bulk on 5 V rail)
[ ]	4	Ceramic capacitor 100 nF (104)	2 for brownout protection as specified + 2 spare
[ ]	1	Electrolytic 10 uF 16 V; 1 uF capacitor	Audio mixer (10 uF) and mic coupling (1 uF)
[ ]	6	Resistors: 2x10k (mixer), 2x100k (solar)	1/4 W, 5%
[ ]	1 m	0.5 mm copper wire (silicone)	ALL high-current paths: battery, switch, regulator, 5 V rail, motors
[ ]	1 set	Dupont wires (F-F, M-F), 20 cm	Signal wiring (SPI, I2C, UART, audio)
[ ]	1+1	Perfboard 5 x 7 cm and 7 x 9 cm	Layer 1 and Layer 2 decks (Layer 2 trimmed to 70 x 65 mm)
[ ]	8	M3 brass standoffs: 4x6mm, 4x25mm	Layer stack (Phase 6) + M3 x 6 screws x 16
[ ]	1	M2.5 / M2 / M3 screw assortment	Self-tap M2.5 x 8 for faceplate, modules, base
[ ]	1	Heat-shrink, Kapton tape, solder	Insulation and joints (60/40 or lead-free 0.8 mm)
[ ]	1	PLA+ or PETG 1 kg spool	Dark colour; PETG recommended for solar lid heat
[ ]	1	Nano-SIM extension cable (optional)	Use if module's SIM tray cannot align with the rear slot

1.5 The 5-Point Field Guide for buying genuine 18650 batteries

Fake cells labelled 9900 mAh are common. A real 18650 does not exceed about 3500 mAh. Apply all five checks:

[PASS] **Weight test:** a genuine 2600-3000 mAh cell weighs **45-48 g**. Under 40 g is fake or very low quality. Weigh them on a kitchen scale (error 1 g).

[PASS] **Capacity label:** reject anything printed above **3500 mAh** (the physical maximum today). Prefer a known brand and a printed discharge rating (10 A or more).

[PASS] **Flat wrapper look:** the plastic wrap should be tight, with clean printing and a crisp insulator ring on the positive end. Wrinkled wrap, blurry text or a tiny positive nipple is a warning sign.

[PASS] **Multimeter voltage check:** new cells should read **above 3.5 V** on arrival (typically 3.6-3.9 V). A cell at 0-2 V is dead or abused. Buy two cells whose voltages are within 0.05 V of each other (required for parallel).

[PASS] **Original laptop pull cells:** a trusted option. Open a good laptop pack, take cells that hold above 3.5 V and test capacity on a charger-tester. Use matched cells only and never mix old and new.

1.6 Power configuration decision (choose A or B now)

	Config B: 1S2P + MT3608 (default in this guide)	Config A: 2S series + 2S BMS + LM2596
Cells	2 x 18650 in PARALLEL: 3.0-4.2 V, double capacity (5200+ mAh)	2 x 18650 in SERIES: 6.0-8.4 V, same capacity as one cell
Charger	TP4056 Type-C (1 cell-voltage charger). Solar via 1N5819 into IN+/IN-. Works exactly as specified	A 1S charger CANNOT charge series cells. Use a 2S Type-C BMS charger board. Solar goes through 1N5819 into its 5 V input
5 V regulator	MT3608 BOOST to 5.00 V (battery is below 5 V)	LM2596 BUCK to 5.00 V (battery is above 5 V)
L298N motor supply (VS)	From the 5 V rail: motors see about 3 V (L298N drops about 2 V). Slow but OK for 3-6 V geared motors	Directly from the 7.4 V battery: motors see about 5.5 V. Much stronger
Pros	Solar charging works with the TP4056 as specified. Simple, longest run time	Strong motors, 3 A regulator, no boost-converter overload on motor start
Cons	Boost converter limited (~1 A at 5 V output from a 3.7 V cell): lower PWM speed, keep 1000 uF caps. Motors weaker	Solar panel only helps if it can reach the charger input (5-6 V panel into 5 V input works). Needs a 2S BMS board

[NOTE]

Recommendation: choose **B** for the solar-charging demonstration the specification asks for. Choose **A** if the robot must drive on carpet or carry weight. Every wiring step in Phase 3 is labelled [A] or [B] where they differ. Everything else (signals, firmware, cube) is identical.

PHASE 2 — Component Dimensions & 3D CAD Modeling / Printing Guide

2.1 Component dimension table (L x W x H, mm, typical)

Part	L x W x H	Notes / re-measure
ESP32 NodeMCU DevKit V1 (30 pin)	51.5 x 28.5 x 13	38-pin boards are 55 mm long. Re-measure
ESP32-CAM (with OV2640)	40 x 27 x 12	Lens centre is on the long edge. Mounts landscape
ESP32-CAM-MB shield	40 x 27 x 8	Used on the bench only for flashing
1.8" ST7735 TFT PCB (with SD slot)	57 x 35 x 6.5	Visible area 35.0 x 28.0. Mount landscape (57 wide)
SIM7600E-H breakout	about 55 x 40 x 9	Largest variation between sellers. Measure
L298N driver	43 x 43 x 27	Mounts holes about 37 x 37 mm. Sits in the base tray
TP4056 Type-C board	26 x 17 x 4.5	USB-C shell about 9.0 x 3.3 mm
MT3608 boost module	36 x 17 x 14	Config B
LM2596 buck module	43 x 21 x 14	Config A
PAM8403 module	21 x 15 x 4	
MAX9814 module	25 x 17 x 6	Mic capsule position varies; pinhole is above it
VL53L0X GY-530 / VL53L1X	25 x 10.5 x 3	Sensor window about 4.4 x 2.4 mm
18650 cell	65 x 18.4 dia (46 g)	Genuine 2600-3000 mAh
2-slot 18650 holder	77 x 39 x 19	Goes in base tray
Speaker 8 ohm 1 W	28 dia x 5	Or 20 mm. Goes behind side-wall grille
Solar panel	60 x 60 x 3 or 70 x 70 x 3	Fits 2 mm pocket (edge glue)
Mini rocker switch (snap-in)	cutout 15 x 10, 20 deep	Measure yours
Tank track chassis plate	about 120 x 90 (kit)	Base tray mounts to its screw holes
Perfboards	70 x 50 and 70 x 65	
Brass standoffs M3	6 mm and 25 mm	Print bosses with 2.5 mm pilot hole

[NOTE]

Re-measure before printing: ESP32-CAM width, TFT PCB length, SIM7600 board size, switch cutout and the tank chassis plate holes. If any part is more than 1 mm larger, scale the matching feature in CAD by the difference.

2.2 The 4 modular printed parts (summary)

#	Part	Outer size	Purpose
1	Base track frame (tray)	120 x 90 x 30	Bolts onto the tank chassis; holds 2 motors' mounts, caster, battery holder, L298N. Lowest centre of gravity
2	Main cubic body shell	80 x 80 x 85 (walls 2.5)	Open top and front. Holds the M3 bosses for Layer 1 and 2, the rear column, speaker grille
3	Removable front faceplate	80 x 85 x 2.5 (+ 2 mm lip)	TFT window, camera lens, mic pinhole, ToF window
4	Top solar lid	80 x 80 x 4	2 mm recessed pocket for the solar panel + 3 mm wire hole

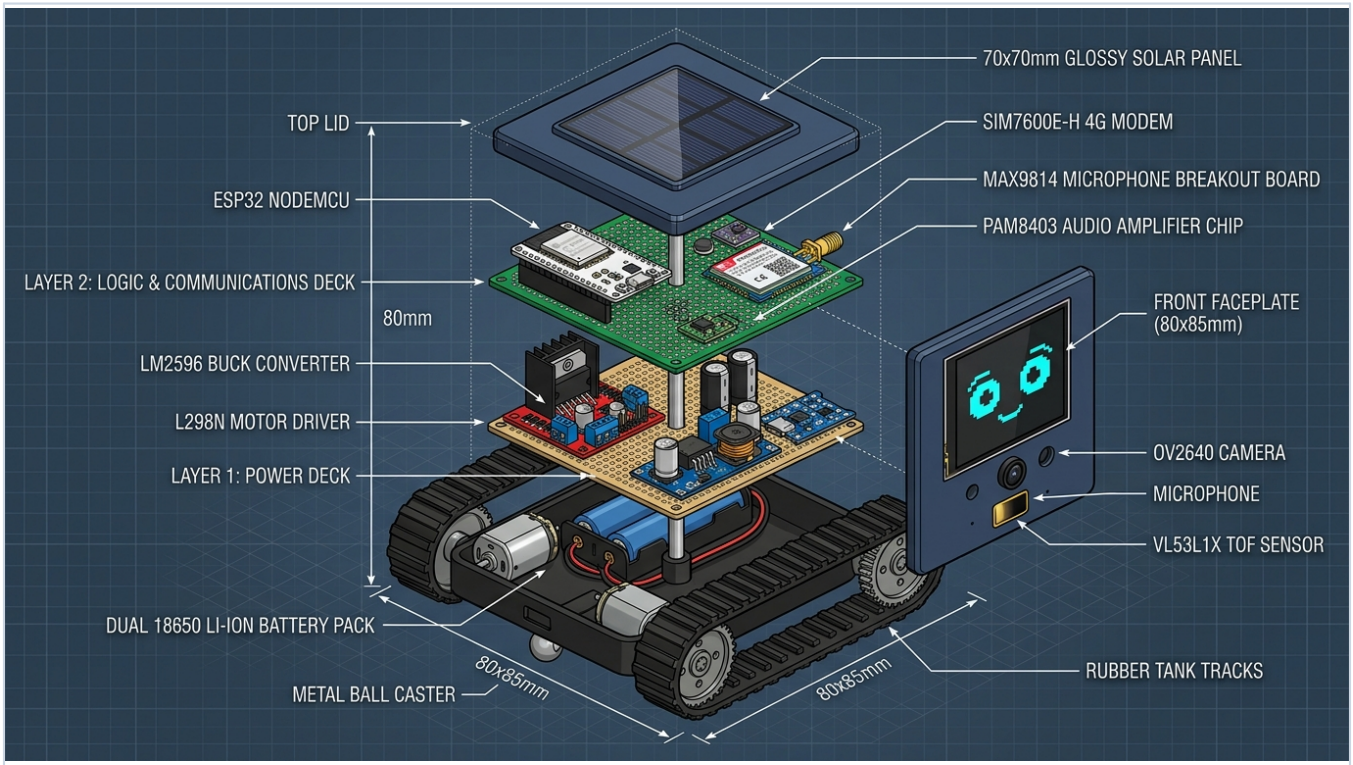


Figure 2.1 — Exploded 3D CAD View: 4 Modular Assembly Tiers (Base Track Tray, Layer 1 Power, Layer 2 Logic/Comms, Front Faceplate & Top Solar Lid).

2.3 Part 1 — Base Track Frame (Fusion 360 / SolidWorks)

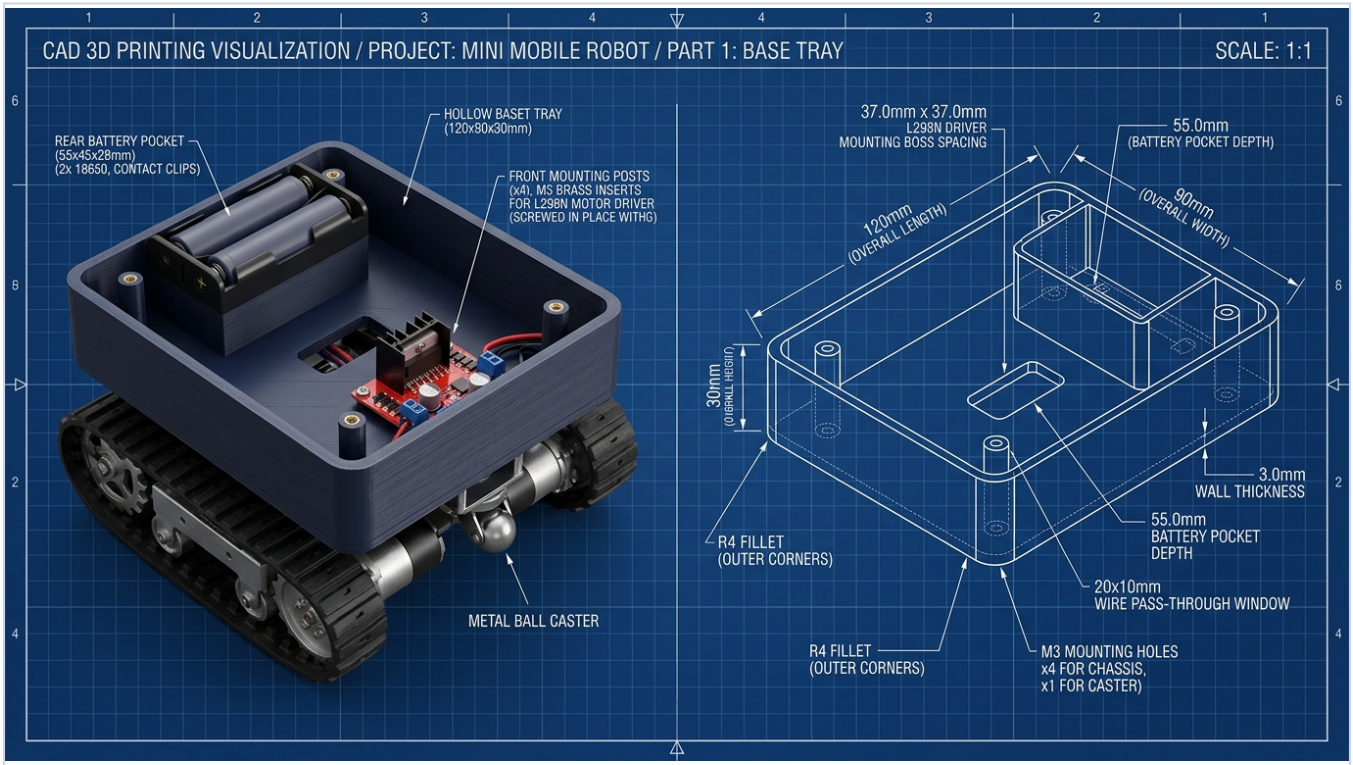


Figure 2.2 — Part 1: Tracked Base Chassis & Battery Tray (120x90x30mm) 3D Render & Engineering CAD Blueprint.

- [ ] **Sketch 1 (Top plane):** rectangle **120 x 90 mm**. Extrude **30 mm**. Fillet vertical edges R4.
- [ ] **Shell:** select the top face, shell inward **3 mm** walls, floor 3 mm (open top). Result: inner bay 114 x 84 x 27.
- [ ] **Chassis-mount holes:** 4 x dia 3.5 mm holes in the floor, aligned to the screw holes of your tank chassis kit plate.
- [ ] **Floor wire window:** cut a **20 x 10 mm** rectangular hole in the centre of the floor so motor wires and battery leads pass into the upper cube.

[ ] **Motor mounts:** most tank kits carry the motors on the kit plate itself. If yours attaches them to the kit, nothing else is needed. Cut two 25 x 12 mm slots in the side walls so the motor wires pass out.

[ ] **Caster mount:** two M3 holes, centre spacing as per your caster (typically 20 or 25 mm), at the rear centreline of the floor.

[ ] **Battery holder bay:** at the REAR of the tray draw a 78 x 40 mm pocket outline (+1 mm). Add 2 x M3 screw bosses on its centreline to screw the holder down (match your holder holes).

[ ] **L298N bay:** forward of the batteries draw four posts (dia 6, pilot 2.5) on a 37 x 37 mm square centred in the front half. Height 5 mm.

[ ] **Export** STL (binary, mm).

2.4 Part 2 — Main Cubic Body Shell 80 x 80 x 85 mm, walls 2.5 mm

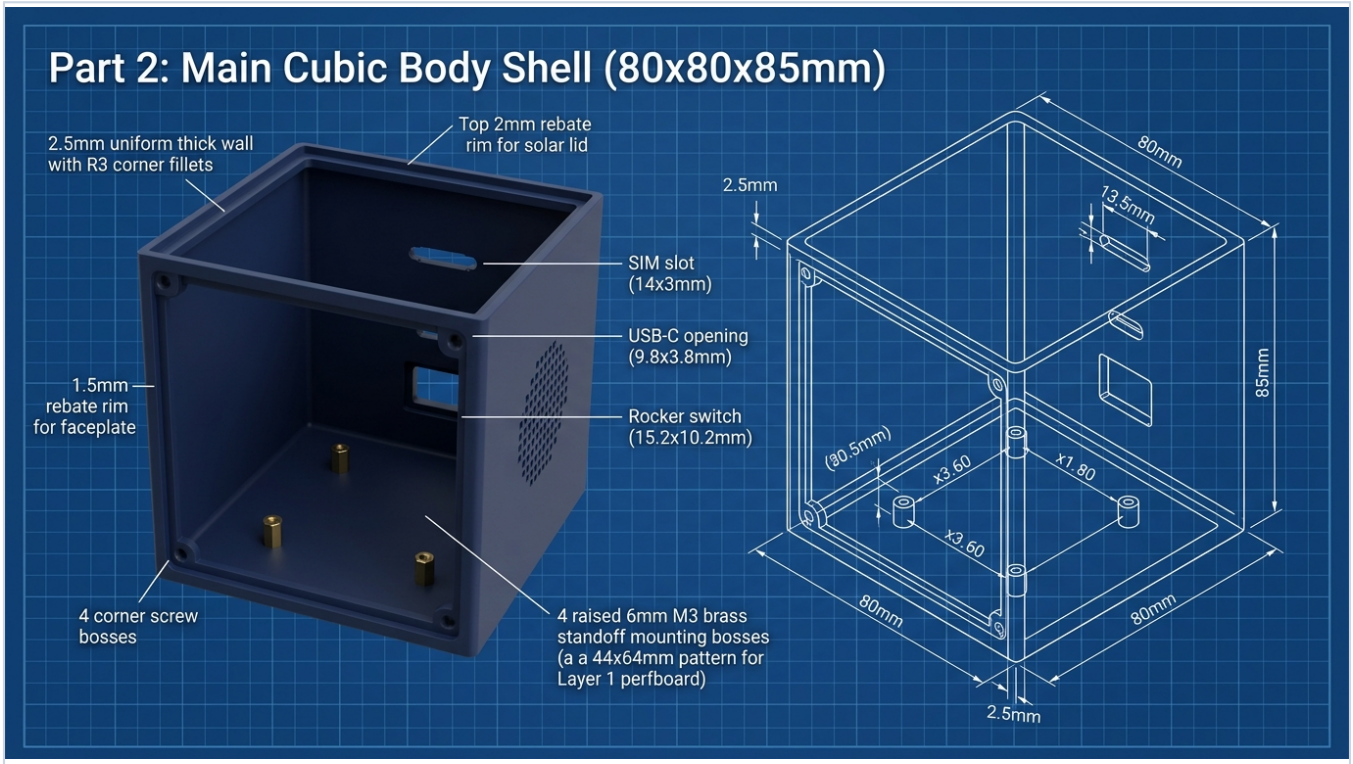


Figure 2.3 — Part 2: Main Equipment Cubic Core Shell (80x80x85mm) 3D Render & Engineering CAD Blueprint.

[ ] **Sketch:** 80 x 80 mm square on the top plane. Extrude **85 mm** (X = width, Y = depth, Z = height). Fillet the four vertical edges R3.

[ ] **Shell:** remove the TOP face and the FRONT face, wall 2.5 mm. Floor remains 2.5 mm. Inner volume 75 x 75 x 82.5 (interior depth 75 up to the faceplate).

[ ] **Faceplate seat:** cut a 1.5 mm x 2 mm rebate round the front opening so Part 3 sits flush. Add 4 corner posts (dia 6, pilot dia 2.5 mm, 8 mm deep) for the faceplate screws at 5 mm from each outer edge.

[ ] **Lid seat:** cut a 2 mm x 1.5 mm rebate around the top edge for Part 4. Add 4 corner screw bosses at the top (dia 6, pilot 2.5 mm).

[ ] **Floor holes:** 4 x dia 3.2 through the floor at 4 mm from corners (cube-to-tray M3). Add the 20 x 10 mm wire window at centre.

[ ] **Layer 1 bosses (power):** 4 cylinders dia 7, **height 6 mm** from the floor, hole dia 2.5 mm pilot for M3 standoffs, on a **44 x 64 mm** rectangle (fits 5 x 7 perfboard, which has corner holes about 3 mm in). Board runs 70 mm across the width, 50 mm deep, front edge 14 mm behind the faceplate (so the TFT fits).

[ ] **Layer 2 bosses:** the 25 mm standoffs stack on the Layer 1 screws (see Phase 6), so no extra bosses are required. For a 70 x 65 board, add 4 holes in the board at 62 x 57 mm and use the same stack: use 4 x M3 x 25 standoffs on the Layer 1 board corners and screws through Layer 2 into them.

[ ] **Rear vertical column (centre X = 40 mm; all positions from the top edge, see Diagram 1 View 2):** (1) SIM slot 14 x 3 mm centred at **17 mm**; (2) ONE USB Type-C cut 9.8 x 3.8 mm centred at **42 mm**; (3) rocker switch cut 15.2 x 10.2 mm centred at **66 mm**. Strictly one column, nothing else on the centre line.

[ ] **TP4056 rear bosses:** on the inner rear wall, 2 posts dia 4, pilot dia 1.9, at the hole spacing of your TP4056 board (typically 17 mm apart vertically), centred so the Type-C shell lines up with the 9.8 x 3.8 cut. Test-fit before final print.

[ ] **Speaker grille:** LEFT side wall: grid of 4 x 3 dia 2.5 holes at 4.5 mm pitch centred 30 mm from the front and 25 mm from the floor. Add a 4-post speaker ring (dia 29 recess, 1 mm deep) behind it. Rear grille: two groups of 2 x 4 holes dia 2.0 at x = 10 and x = 70 mm, 57-72 mm from the top.

[ ] **4G antenna area:** keep the right inner wall free (50 x 20 mm flat) to stick the FPC antenna.

[ ] **Export** STL.

2.5 Part 3 — Removable Front Faceplate (+0.4 mm tolerance on every cut)

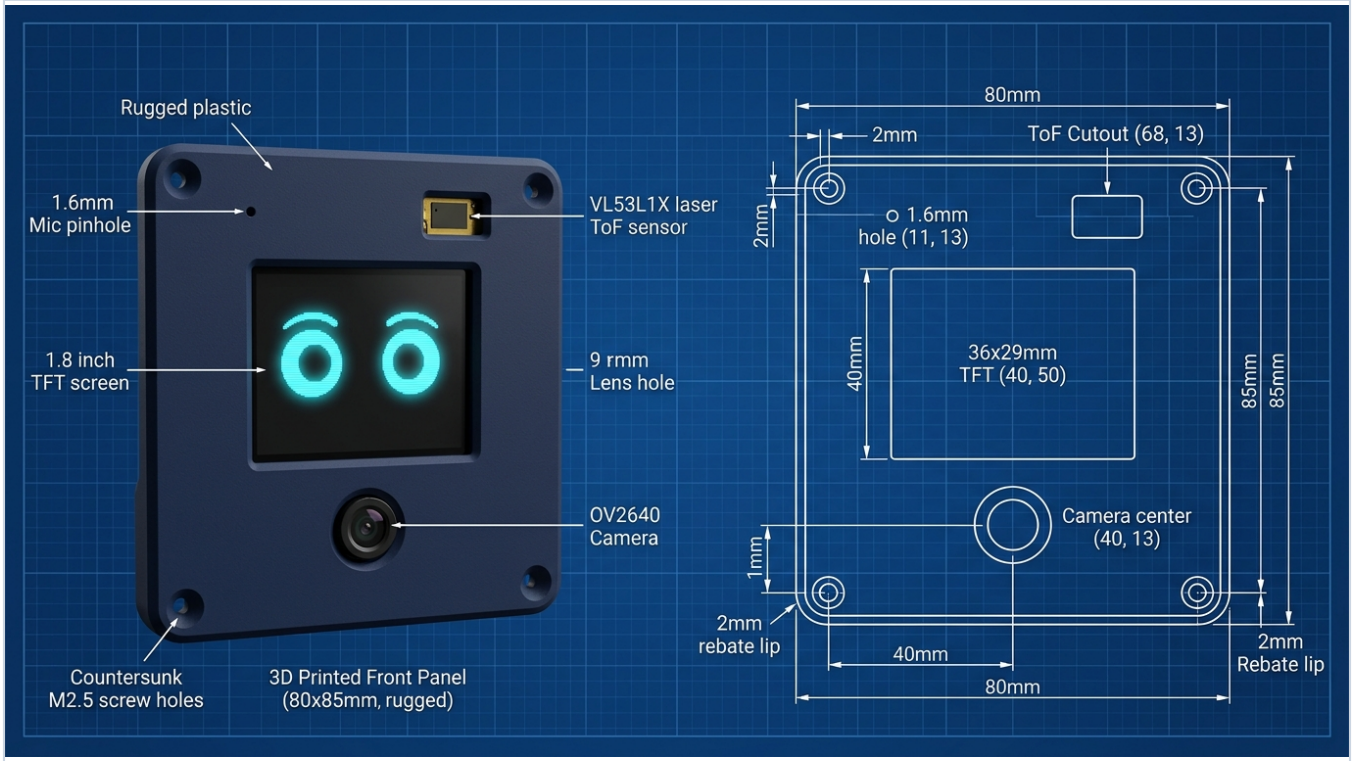


Figure 2.4 — Part 3: Front Sensory Bezel & Faceplate (80x85x12mm) 3D Render & Engineering CAD Blueprint.

[ ] **Sketch:** 80 x 85 mm, extrude 2.5 mm. Add a 2 mm lip (inset 2.5 mm, matching the body rebate).

[ ] **TFT window:** rectangle centred at (40, 50) measured from the top-left of the outer face. Visible area is 35.0 x 28.0, so cut **36 x 29 mm** (this is the area plus 0.4 and a small bezel margin). Screw posts for the PCB: 4 posts dia 5 on your TFT hole pattern (about 52 x 30 mm), pilot 1.9 mm, standing 6 mm proud on the inside.

[ ] **Camera lens hole:** dia **9.0 mm** (lens barrel about 8.2 + 0.4 clearance + chamfer) centred at (40, 13). 2 mm posts or a printed clip hold the ESP32-CAM board landscape behind it. Check that the lens barrel centre is on the board's edge you mount against the plate.

[ ] **Mic pinhole:** dia **1.6 mm** at (11, 13). Add a 4 mm recess on the inside so the MAX9814 capsule seats against the hole with a thin foam ring.

[ ] **ToF window:** rectangle **10 x 6 mm** at (68, 13). VL53L0X/L1X must see through a clean opening: do NOT cover it with plastic or glass; leave a free hole.

[ ] **Corner screw holes:** 4 x dia 2.8 at (5,5), (75,5), (5,80), (75,80). Countersink 0.8 mm.

[ ] **Export** STL (print face-down, the outer face on the bed).

2.6 Part 4 — Top Solar Lid

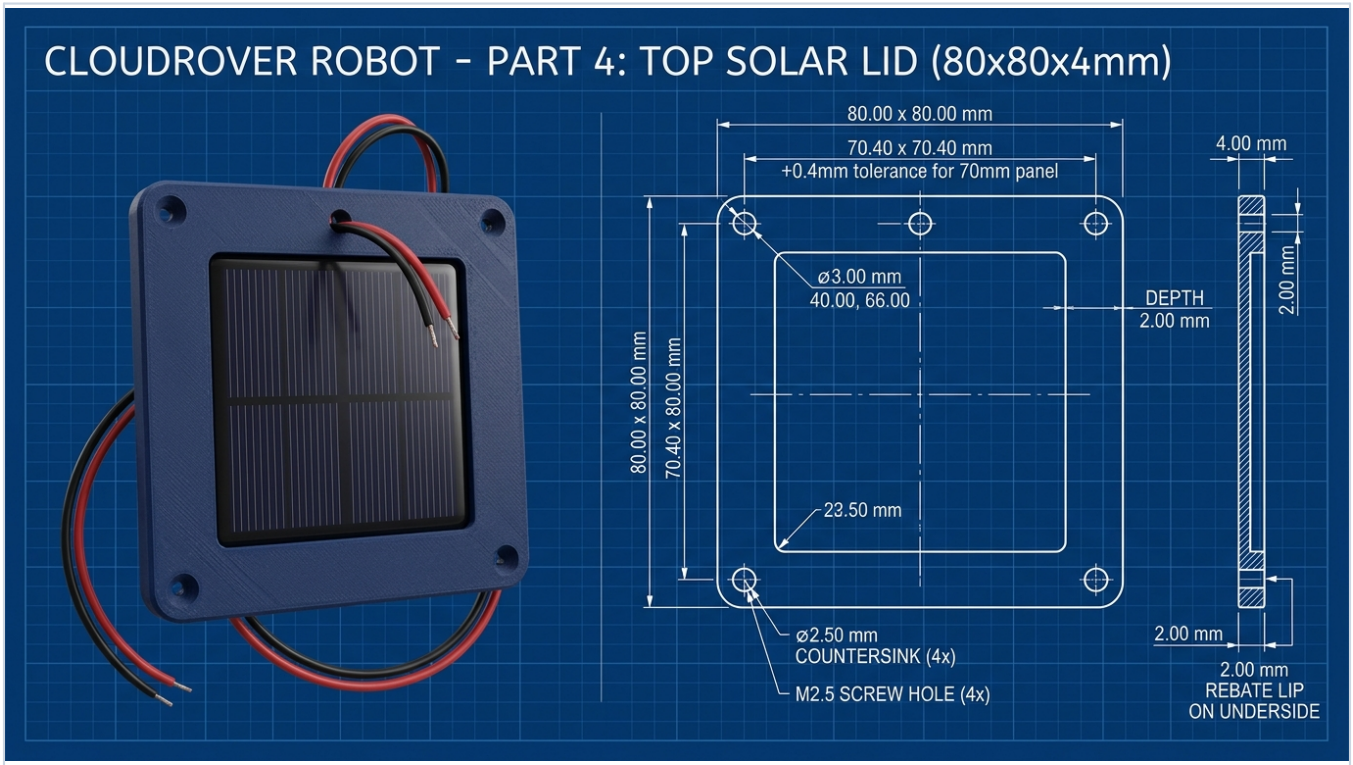


Figure 2.5 — Part 4: Top Solar Canopy & Weather Lid (80x80x4.5mm) 3D Render & Engineering CAD Blueprint.

- [ ] **Sketch:** 80 x 80 mm, extrude **4 mm** with the 2 mm rebate lip underneath (lid seat).
- [ ] **Pocket:** centre a square pocket **70.4 x 70.4 mm** (for a 70 mm panel) or **60.4 x 60.4 mm** (60 mm panel), **2.0 mm deep** (top surface so the panel sits flush: panel is 2-3 mm thick; if 3 mm let it stand 1 mm proud, that is fine).
- [ ] **Wire hole:** dia **3 mm** through the pocket floor at (40, 66), inside the pocket, near the rear edge.
- [ ] **Corner holes:** 4 x dia 2.8 into the body bosses at the lid corners (outside the pocket: 80 - 70.4 = 4.8 mm per side, so use countersunk dia 2.5 screws at (2.4, 2.4) or reduce to the 60 mm pocket if holes clash).
- [ ] **Export** STL.

2.7 Slicer settings (Cura / PrusaSlicer / Bambu Studio)

Setting	Value	Setting	Value
Material	PLA+ (or PETG for the lid and body)	Nozzle / line width	0.4 mm / 0.42 mm
Layer height	0.2 mm	First layer	0.2-0.28 mm
Walls / perimeters	3 (0.42 x 3 = 1.26 mm)	Top/bottom layers	5 / 4
Infill	25 %, gyroid or grid	Nozzle temperature	PLA+ 205-215 C; PETG 235-245 C
Bed	PLA+ 60 C; PETG 80 C	Print speed	50 mm/s outer walls 30
Supports	Part 1: none. Part 2: ON ('touching buildplate' only) for rear column cuts and side grille; Part 3/4: none	Adhesion	Brim 5 mm for parts 2 and 3
Orientation	Part 1 upright; Part 2 upright with open top up; Part 3 face down; Part 4 pocket UP	Cooling	Fan 100 % PLA+, 40 % PETG
Estimated time	About 3 h + 5 h + 1 h + 1.5 h	Material	About 130 g total

[NOTE]

First-print check: before printing the big shell, print a 20 x 40 mm test coupon with the USB-C (9.8 x 3.8) and rocker (15.2 x 10.2) cut-outs and push your real parts into it. Adjust the cut-outs by the measured difference. This saves a failed 5-hour print.

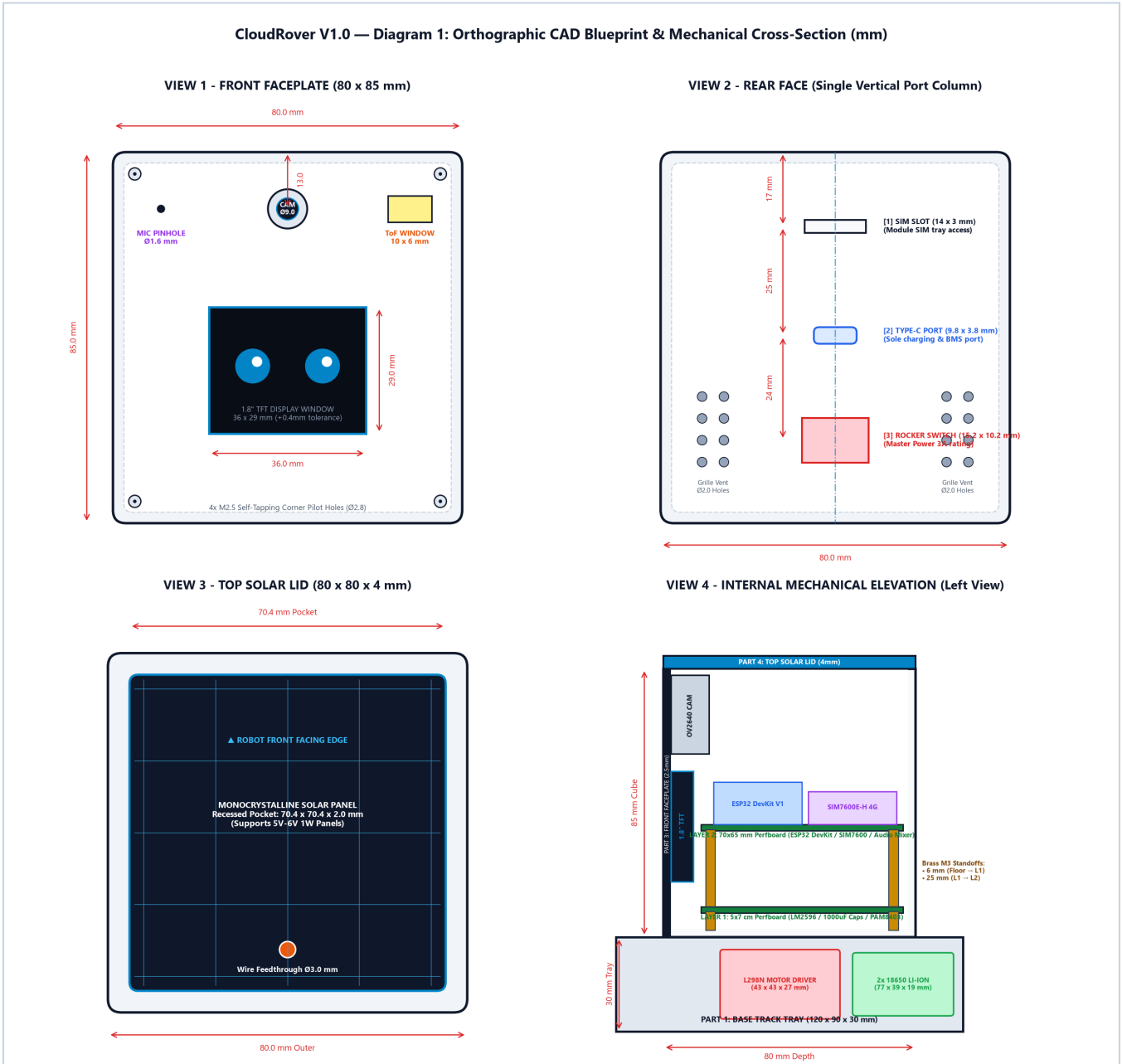


Figure 2.6 — Diagram 1: Orthographic 4-View CAD Blueprint & Mechanical Cross-Section (Dimensions in mm).

PHASE 3 — Bench Calibration & Power Distribution Assembly

[CRITICAL SAFETY WARNING]

Do NOT connect the ESP32, ESP32-CAM, SIM7600E-H or any other logic board until the regulator output has been set to exactly 5.00 V (Step 3.2).

Tools: soldering iron 350 C, multimeter, wire strippers, flush cutters, tweezers, heat-shrink, Kapton tape, helping hands, fume extractor/open window.

Step 3.1 — Solder the 1N5819 Schottky diode to the solar panel and TP4056 IN+ / IN-

- [ ] Identify the panel polarity with the multimeter in DC volts in daylight (red lead on + wire gives a positive reading). Mark with a red marker.
- [ ] Cut the solar red lead to 12 cm. Pre-tin the stripped ends.
- [ ] Cut the diode leads to 8 mm. Identify the cathode: it has the **silver ring** painted round one end.
- [ ] Solder the panel's red wire to the diode's ANODE (the unmarked end). Slide heat-shrink over the joint and shrink it.
- [ ] Solder the diode's CATHODE (silver ring end) to the TP4056 board's **IN+** pad (use 0.5 mm wire if extending it).
- [ ] Solder the solar panel's black wire directly to the TP4056 **IN-** pad.
- [ ] **Test:** take the panel near a window. Measure DC volts on the TP4056 IN+ and IN- pads: you should see 5-6 V. Cover the panel: voltage drops. The diode prevents the battery from draining back into the panel at night.

Step 3.2 (CRITICAL) — Set and calibrate the regulator to exactly 5.00 V

[B] Config B: 1S2P (default)

- [ ] Take the 2-slot 18650 holder (parallel: red to both positive terminals, black to both negatives). Solder red to TP4056 **B+** and black to **B-**.
- [ ] Solder a 0.5 mm red wire from TP4056 **OUT+** to one terminal of the rocker switch.
- [ ] Solder a 0.5 mm red wire from the other switch terminal to the MT3608 boost module **VIN+** pad. Solder a 0.5 mm black wire from TP4056 **OUT-** to MT3608 **VIN-**.

[A] Config A: 2S

- [ ] Take the 2-slot 18650 series holder (cells in series, red is 7.4 V +, black is GND). Solder to your 2S BMS board battery pads.
- [ ] From the switched output of the BMS, route red through the rocker switch to LM2596 **IN+**, black to LM2596 **IN-**.

Calibrate the regulator to 5.00 V (both configs)

- [ ] Insert the 18650 cells. Check polarity twice (the flat end of the cell goes against the holder's spring).
- [ ] Attach multimeter clips to the regulator's **OUT+** and **OUT-** pads, set to DC 20 V.
- [ ] Switch the rocker ON.
- [ ] Turn the multi-turn brass trimmer screw on the regulator with a small flat screwdriver until the meter reads **5.00 V (between 4.95 V and 5.05 V)**. Counter-clockwise usually raises the voltage on MT3608; turn 10-20 turns if nothing moves.
- [ ] Switch the rocker OFF and ON three times. Confirm the meter returns to 5.00 V every time.
- [ ] Paint a drop of nail polish or hot glue on the trimmer screw so vibration cannot move it. **Never connect logic boards until this is done.**

Step 3.3 — Build the 5 x 7 cm perfboard power rail (5 V, 3.3 V, GND)

- [ ] Cut a piece of 0.5 mm solid bare copper wire (or strip 0.5 mm silicone wire) to 60 mm. Solder it flat across one long row of holes on the copper side. This is the **5 V HIGH-CURRENT RAIL**.
- [ ] Solder a second 60 mm wire parallel to it, two rows away. This is the **COMMON GND RAIL**.
- [ ] Solder a third 40 mm wire next to GND. This is the **3.3 V RAIL**.
- [ ] Solder the regulator OUT+ wire (0.5 mm red) to one end of the 5 V rail, and OUT- (0.5 mm black) to the GND rail.
- [ ] Solder a **1000 uF 16 V electrolytic capacitor** directly across the 5 V and GND rails (long lead = positive to 5 V rail, white/gold stripe lead = negative to GND rail). Solder a **100 nF ceramic capacitor** right next to it. These

absorb the 2 A bursts when the 4G modem transmits and the camera takes a picture.

[ ] Solder the **PAM8403** module flat to one corner of this perfboard (it runs from 5 V). Connect its +5V pad to the 5 V rail, GND to GND rail.

[ ] Test with the multimeter in ohms: 5 V rail to GND rail must read very high (megaohms, rising as the 1000 uF charges). A zero-ohm reading means a solder bridge: find and fix it now.

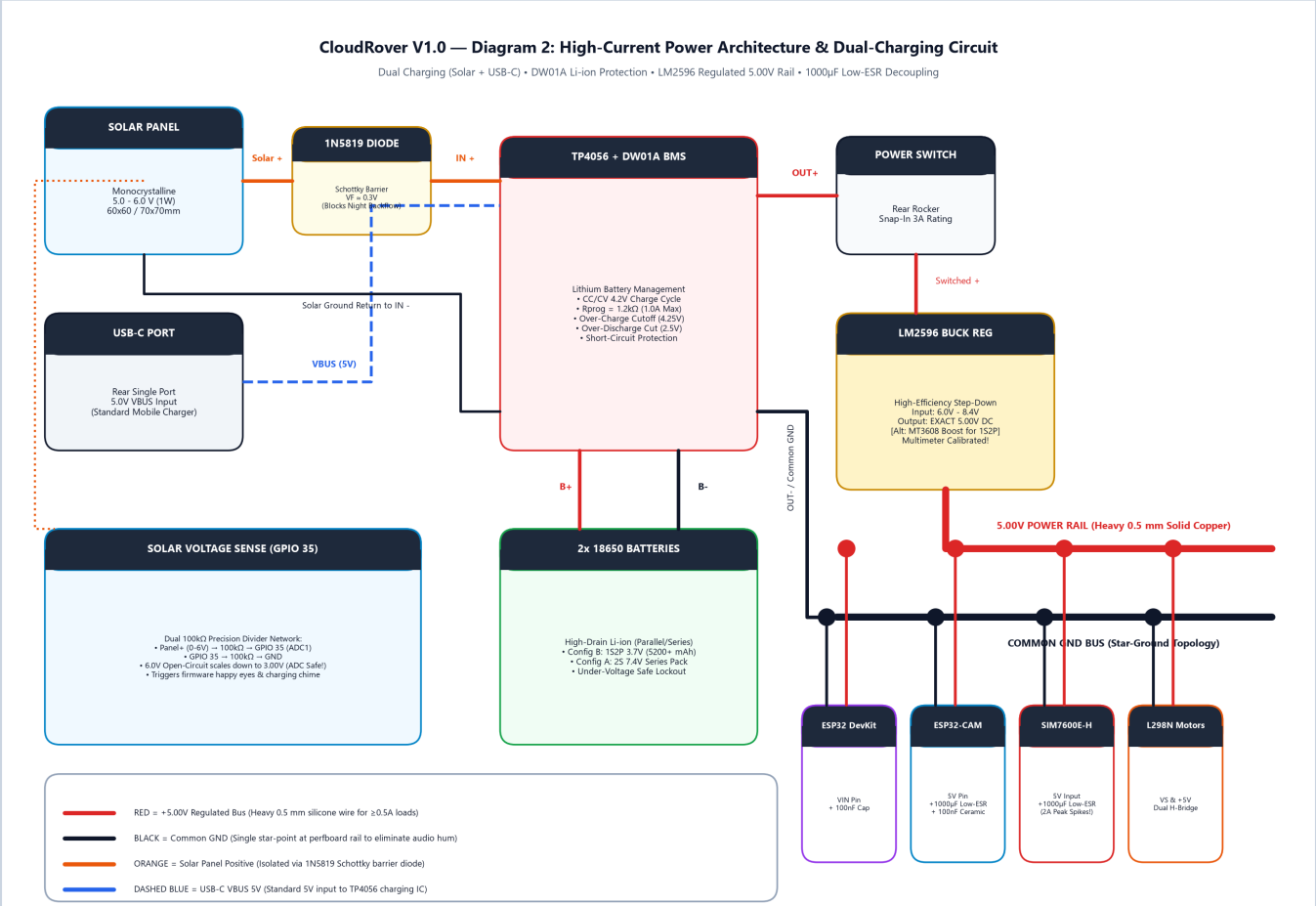


Figure 3.1 — Diagram 2: High-Current Power Architecture & Dual-Charging Circuit (Solar + Type-C, LM2596 Regulated 5.00V Rail, 1000µF Low-ESR Decoupling).

PHASE 4 — Pin-by-Pin Signal & Audio Wiring

4.1 Master pinout table (ESP32 NodeMCU DevKit V1)

ESP32 pin	Connects to	Signal	Wire colour
GPIO 5	TFT CS	Chip select (SPI)	Blue
GPIO 4	TFT RST	Reset	Blue
GPIO 2	TFT DC / A0	Data/Command	Blue
GPIO 23	TFT SDA / MOSI	Master Out Slave In (SPI)	Blue
GPIO 18	TFT SCK	Clock (SPI)	Blue
GPIO 21	VL53L0X / VL53L1X SDA	I2C data (has on-board pull-up)	Green
GPIO 22	VL53L0X / VL53L1X SCL	I2C clock	Green
GPIO 16 (RX2)	SIM7600 TXD	UART2 receive (crossed!)	Cyan
GPIO 17 (TX2)	SIM7600 RXD	UART2 transmit (crossed!)	Cyan
GPIO 12	L298N IN1	Left motor dir A (see warning below)	Red
GPIO 13	L298N IN2	Left motor dir B	Red
GPIO 14	L298N IN3	Right motor dir A	Red
GPIO 27	L298N IN4	Right motor dir B	Red
GPIO 32	L298N ENA	Left PWM speed	Orange
GPIO 33	L298N ENB	Right PWM speed	Orange
GPIO 34	MAX9814 OUT (and SIM7600 MIC+ via 1 uF)	ADC1 input-only; clap detection	Purple
GPIO 25 (DAC1)	10 uF + 10 k -> mixer -> PAM8403 L_IN	Sound effects synthesis	Yellow
GPIO 35	Solar sense divider (2 x 100 k)	ADC1 input-only; detects sunlight for chime	Orange
VIN (5 V)	5 V rail	Regulated Logic Power	Red (0.5 mm)
GND	GND rail	Common star ground	Black (0.5 mm)

[CRITICAL SAFETY WARNING]

GPIO 12 is a boot-strap pin (selects flash voltage). Keep the L298N IN1 wire **disconnected while uploading** if the upload fails with 'invalid header' or 'flash voltage' errors, and never add a pull-up to it. With ENA/ENB low at boot the L298N pulls the input low, which is the safe state.

[NOTE]

Unused / free ESP32 pins you can keep for expansion: 0 (boot button, do not use), 15, 19, 26, 36 (VP), 39 (VN). GPIO 6-11 are the internal flash and must never be used.

4.2 Sequential wiring checklist

A. Power taps (use 0.5 mm wire)

[ ] **Step 1:** Perfboard 5 V rail to ESP32 NodeMCU **VIN** pin; GND rail to ESP32 GND.

[ ] **Step 2:** 5 V rail to **L298N +5V** (logic) AND **L298N VS** [B]. For [A] connect VS to the SWITCHED battery +. GND rail to L298N GND. **Remove the ENA and ENB jumpers** (otherwise PWM speed control does not work). If a 5 V-EN jumper exists, remove it (VS is not above 12 V).

[ ] **Step 3:** 5 V rail (via 1000 uF cap, already soldered) to **SIM7600E-H 5 V**; GND to its GND. Use the 1000 uF + 100 nF closest to the module.

[ ] **Step 4:** 5 V rail (via 1000 uF cap #2) to ESP32-CAM **5V** pin; GND to ESP32-CAM GND. (No data wires between the two ESP32s.)

[ ] **Step 5:** 5 V rail to **PAM8403 +5V**; GND to PAM8403 GND.

[ ] **Step 6:** ESP32 **3V3** pin to the 3.3 V rail; 3.3 V rail to TFT VCC and LED, ToF VIN and MAX9814 VDD.

B. TFT, ToF and mic signals (Dupont)

- [ ] **Step 7:** TFT CS -> GPIO 5, RST -> GPIO 4, DC/A0 -> GPIO 2, SDA/MOSI -> GPIO 23, SCK -> GPIO 18, GND -> GND, VCC and LED -> 3.3 V rail. Leave the SD pins (SD_CS, SD_MISO etc.) unconnected.
- [ ] **Step 8:** VL53L0X/L1X SDA -> GPIO 21, SCL -> GPIO 22, VIN -> 3.3 V, GND -> GND. Leave XSHUT/GPIO1 unconnected.
- [ ] **Step 9:** MAX9814 VDD -> 3.3 V, GND -> GND, **GAIN pin: leave floating (60 dB) or tie to GND for 50 dB if clapping saturates**, AR floating. OUT -> GPIO 34 (direct).
- [ ] **Step 10:** Solar sense: panel + (after the 1N5819, at TP4056 IN+) -> 100 k -> **GPIO 35**; GPIO 35 -> 100 k -> GND. At 6 V open-circuit the pin sees 3.0 V (safe).

C. Motors and 4G

- [ ] **Step 11:** L298N IN1 -> GPIO 12, IN2 -> GPIO 13, IN3 -> GPIO 14, IN4 -> GPIO 27, ENA -> GPIO 32, ENB -> GPIO 33.
- [ ] **Step 12:** Left motor to OUT1/OUT2, right motor to OUT3/OUT4 (swap the two wires later if a track runs backwards: Phase 7 table).
- [ ] **Step 13:** SIM7600 **TXD -> GPIO 16**, SIM7600 **RXD -> GPIO 17** (crossed!). Set the module's UART level jumper to 3.3 V if present. GND common.
- [ ] **Step 14:** Insert the SIM, attach the FPC antenna (or SMA adapter + antenna) before powering the module. Never power a 4G module without an antenna attached.

4.3 Audio wiring

- [ ] **Step 15:** MAX9814 OUT -> **1 uF capacitor** -> SIM7600 **MIC+** (and MIC- to GND). The 1 uF blocks the 1.25 V DC bias from the module's mic input.
- [ ] **Step 16:** The same MAX9814 OUT also goes (no capacitor) to GPIO 34. The ADC sees a centred 1.25 V signal: peak-to-peak is used in firmware so the DC does not matter.
- [ ] **Step 17:** GPIO 25 (DAC1) -> **10 uF (+ side to GPIO 25)** -> **10 k** -> mixer node.
- [ ] **Step 18:** SIM7600 **EAR+** -> **10 k** -> the same mixer node. (EAR- to GND.)
- [ ] **Step 19:** Mixer node -> **PAM8403 L_IN**. PAM8403 R_IN to GND. (If your PAM module has input caps built in, nothing else is required.)
- [ ] **Step 20:** PAM8403 L_OUT+ and L_OUT- -> the 8 ohm speaker (not to GND!). Class-D outputs are bridge-tied, never ground one speaker terminal.
- [ ] **Step 21:** Route the audio wires away from the motor and L298N wires (at least 2 cm) and keep them short and twisted with the ground.

4.4 Pre-power inspection checklist

- [ ] No bare strands touching neighbours. Look at every joint with a magnifier.
- [ ] Meter continuity: 5 V to GND and 3V3 to GND must NOT beep.
- [ ] Rocker is OFF. USB cable is NOT yet plugged in.
- [ ] ENA/ENB jumpers removed from the L298N.
- [ ] TXD/RXD crossed correctly (module TXD to GPIO 16).

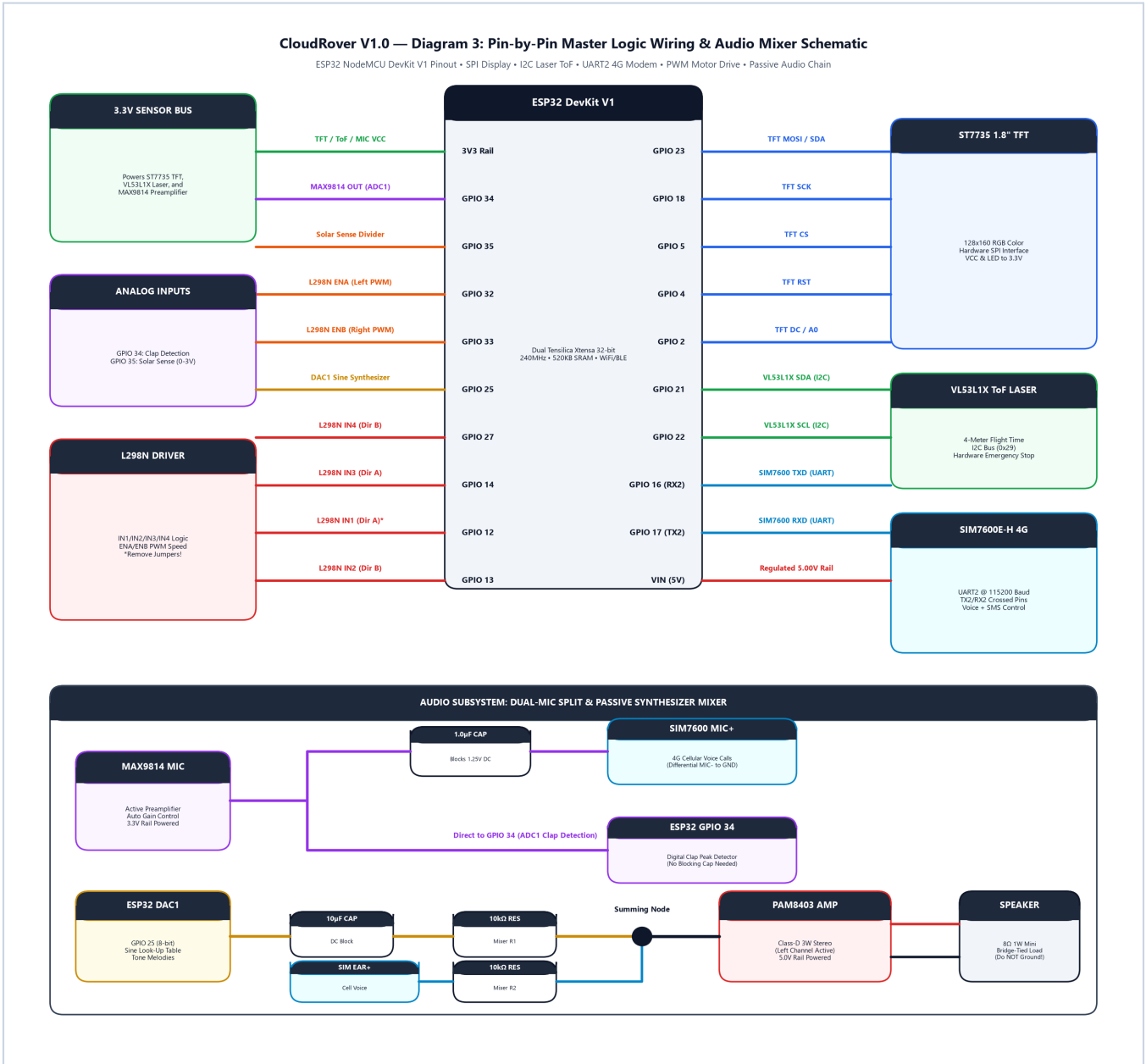


Figure 4.1 — Diagram 3: Colour-Coded Pin-by-Pin Master Logic Wiring & Audio Mixer Schematic.

PHASE 5 — Arduino IDE Setup & Ready-to-Upload Source Code

Step 5.1 — Install the tools

- [ ] Install **Arduino IDE 2.x** (arduino.cc, free).
- [ ] **File -> Preferences -> Additional boards manager URLs**, paste:
  `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- [ ] **Tools -> Board -> Boards Manager**, search 'esp32', install **esp32 by Espressif** (version 2.0.17 or 3.x: both work with this code).
- [ ] **Sketch -> Include Library -> Manage Libraries**, install: **Adafruit GFX Library**, **Adafruit ST7735 and ST7789 Library**, **Adafruit BusIO** (if asked), **Adafruit VL53L0X** (or **VL53L1X by Pololu** if you bought the L1X), **TinyGSM** (optional; the supplied firmware talks to the modem with plain AT commands so it does not need it).
- [ ] Install the USB driver: the DevKit has a CP2102 or CH340 chip. Plug it in; if no COM port appears, install the matching driver.
- [ ] Create two sketch folders: **CloudRover_Main** and **CloudRover_CAM**. Paste the code below into files with the same name + .ino (the folder name must equal the file name). Copies of both are also provided next to this PDF.

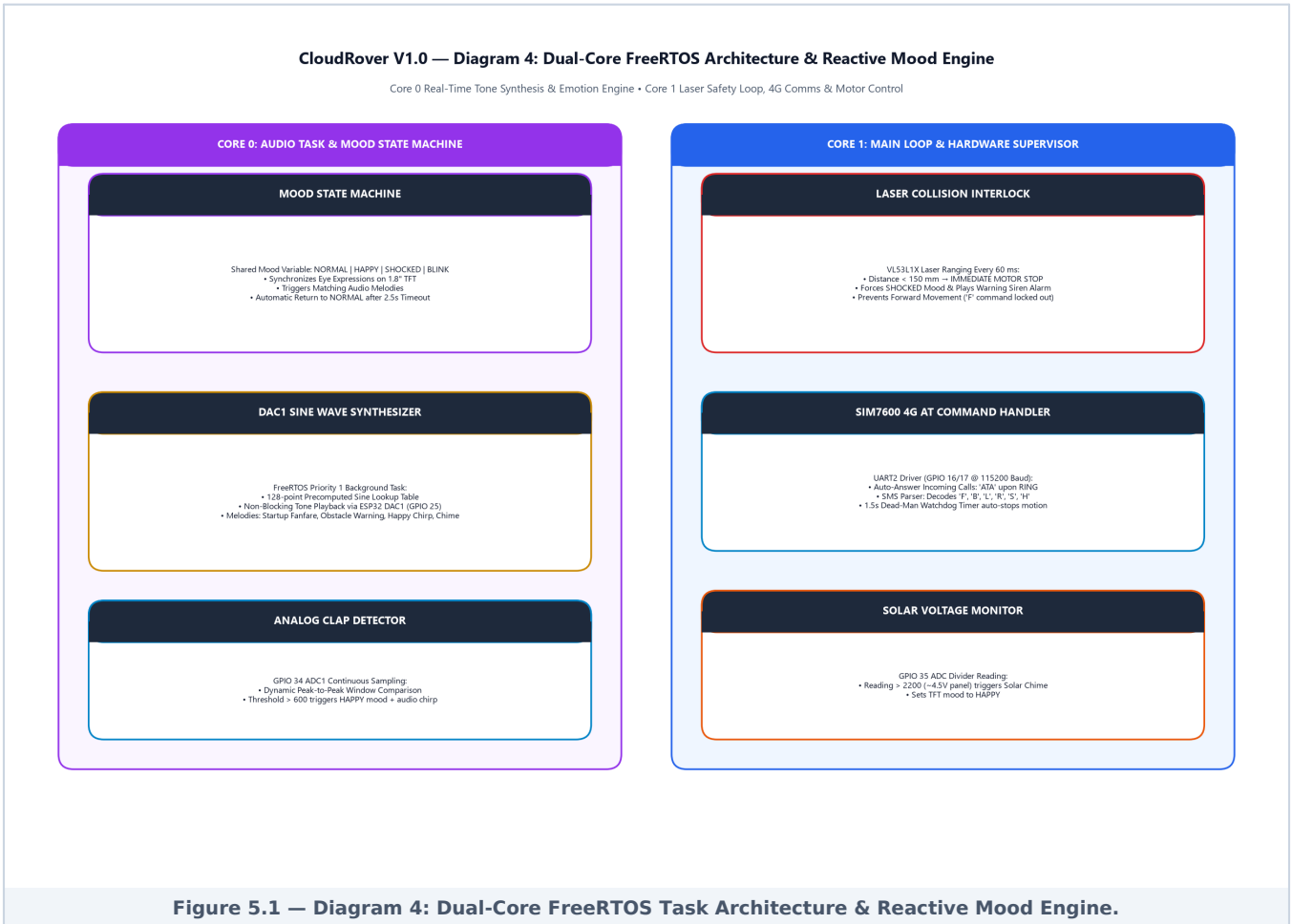


Figure 5.1 — Diagram 4: Dual-Core FreeRTOS Task Architecture & Reactive Mood Engine.

Step 5.2 — Full Code #1: Main ESP32 NodeMCU firmware

Board: 'ESP32 Dev Module'. **Upload speed:** 460800. **If you chose VL53L1X** change `#define USE_VL53L1X 0` to 1. If the colours or the picture offset on the TFT look wrong, change `INITR_BLACKTAB` to `INITR_GREENTAB` (or REDTAB). If the robot turns the wrong way, see Phase 7.

What it does: (1) buffered eye animation with blink, happy, shocked and look-left/right moods; (2) tones through DAC1 from a background FreeRTOS task (non-blocking) for startup, alarm, clap and solar chime; (3) laser distance every 60 ms with an emergency stop and shocked eyes + alarm below 150 mm; (4) clap detection on GPIO 34; (5) L298N PWM motion functions; (6) SIM7600 AT initialisation, auto-answer (ATA) for the intercom, and SMS movement

commands F B L R S and H.

CloudRover_Main.ino (Primary Controller Firmware)

```
 /* CloudRover V1.0 - Main ESP32 NodeMCU (DevKit V1) firmware
 * Board: "ESP32 Dev Module" | Libraries: Adafruit GFX, Adafruit ST7735 and ST7789,
 *        Adafruit VL53L0X (or Pololu VL53L1X if USE_VL53L1X is 1)
 * Eyes: ST7735 160x128 (landscape) | Sound: DAC1 GPIO25 | Motors: L298N | 4G: SIM7600E-H
 */
#include <Wire.h>
#include <SPI.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7735.h>

#define USE_VL53L1X 0               // 0 = VL53L0X, 1 = VL53L1X
#if USE_VL53L1X
  #include <VL53L1X.h>
  VL53L1X tof;
#else
  #include <Adafruit_VL53L0X.h>
  Adafruit_VL53L0X tof;
#endif

// ---------------- PIN MAP ----------------
#define TFT_CS 5
#define TFT_RST 4
#define TFT_DC 2
#define PIN_MIC 34
#define PIN_DAC 25
#define PIN_SOLAR 35                // solar sense: Solar+ -100k- GPIO35 -100k- GND
#define M_IN1 12
#define M_IN2 13
#define M_IN3 14
#define M_IN4 27
#define M_ENA 32
#define M_ENB 33
#define SIM_RX 16                   // ESP RX2 <- SIM7600 TXD
#define SIM_TX 17                   // ESP TX2 -> SIM7600 RXD

// ---------------- SETTINGS ----------------
#define STOP_MM 150                 // emergency stop distance
#define CLAP_LEVEL 900              // peak-to-peak ADC counts (0-4095) = clap
#define DRIVE_SPEED 200             // PWM 0-255
#define CMD_TIMEOUT_MS 1500         // drive command auto-stops after this

Adafruit_ST7735 tft(TFT_CS, TFT_DC, TFT_RST);
GFXcanvas16 cv(160, 128);           // off-screen buffer = flicker-free animation
#define SIM Serial2

// ---------------- AUDIO (FreeRTOS task, non-blocking) ----------------
```

```
enum Snd { SND_NONE, SND_STARTUP, SND_ALARM, SND_CLAP, SND_CHARGE, SND_HAPPY };
QueueHandle_t sndQ;
uint8_t sineTab[256];

void playNote(int hz, int ms) {
  if (hz <= 0) { dacWrite(PIN_DAC, 128); vTaskDelay(ms / portTICK_PERIOD_MS); return; }
  uint32_t period = 1000000UL / hz, t0 = micros(), endT = t0 + (uint32_t)ms * 1000UL;
  while ((int32_t)(endT - micros()) > 0) {
    uint32_t ph = ((micros() - t0) % period) * 256UL / period;
    dacWrite(PIN_DAC, sineTab[ph & 255]);
    delayMicroseconds(25);
  }
  dacWrite(PIN_DAC, 128);
  vTaskDelay(15 / portTICK_PERIOD_MS);
}
void audioTask(void *) {
  Snd s;
  for (;;) {
    if (xQueueReceive(sndQ, &s, portMAX_DELAY) != pdTRUE) continue;
    switch (s) {
      case SND_STARTUP: playNote(523,120); playNote(659,120); playNote(784,120); playNote(1047,300); break;
      case SND_ALARM:   for (int i = 0; i < 3; i++) { playNote(1800,120); playNote(0,60); } break;
      case SND_CLAP:    for (int f = 700; f < 1900; f += 150) playNote(f, 25); break;
      case SND_CHARGE:  playNote(880,100); playNote(1175,100); playNote(1568,200); break;
      case SND_HAPPY:   playNote(988,80); playNote(1319,80); playNote(1568,160); break;
      default: break;
    }
    dacWrite(PIN_DAC, 128);
  }
}
void sound(Snd s) { xQueueSend(sndQ, &s, 0); }

// ---------------- MOTORS ----------------
unsigned long lastDriveCmd = 0;
bool driving = false;
char driveDir = 'S';

void pwmWrite(int pin, int duty) {
#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(pin, duty);
#else
  ledcWrite(pin == M_ENA ? 0 : 1, duty);
#endif
}
void motorsInit() {
  int p[] = {M_IN1, M_IN2, M_IN3, M_IN4};
```

```
  for (int i = 0; i < 4; i++) { pinMode(p[i], OUTPUT); digitalWrite(p[i], LOW); }
#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(M_ENA, 5000, 8); ledcAttach(M_ENB, 5000, 8);
#else
  ledcSetup(0, 5000, 8); ledcSetup(1, 5000, 8);
  ledcAttachPin(M_ENA, 0); ledcAttachPin(M_ENB, 1);
#endif
}
void setMotors(bool l1, bool l2, bool r1, bool r2, int spd) {
  digitalWrite(M_IN1, l1); digitalWrite(M_IN2, l2);
  digitalWrite(M_IN3, r1); digitalWrite(M_IN4, r2);
  pwmWrite(M_ENA, spd); pwmWrite(M_ENB, spd);
}
// If a side runs backwards, swap its two wires on the L298N OUT terminals.
void stopMotors() { setMotors(0,0,0,0,0); driving = false; driveDir = 'S'; }
void forward(int s = DRIVE_SPEED)  { setMotors(1,0,1,0,s); driving = true; driveDir = 'F'; }
void backward(int s = DRIVE_SPEED) { setMotors(0,1,0,1,s); driving = true; driveDir = 'B'; }
void turnLeft(int s = DRIVE_SPEED) { setMotors(0,1,1,0,s); driving = true; driveDir = 'L'; }
void turnRight(int s = DRIVE_SPEED){ setMotors(1,0,0,1,s); driving = true; driveDir = 'R'; }

// ---------------- EYES ----------------
enum Mood { M_NORMAL, M_HAPPY, M_SHOCK, M_CURIOUS };
Mood mood = M_NORMAL;
unsigned long moodUntil = 0, nextBlink = 0, blinkStart = 0, nextGlance = 0;
float lookX = 0, lookTarget = 0;          // -1 (left) .. +1 (right)
const uint16_t C_EYE = 0x07FF;            // cyan
const uint16_t C_BG = 0x0000;

void setMood(Mood m, unsigned long ms) { mood = m; moodUntil = millis() + ms; }

void drawEye(int cx, int cy, int w, int h, int lid, bool happy) {
  // lid: 0..100 percent closed. happy: cheeks push the lower lid up
  int hh = h * (100 - lid) / 100; if (hh < 4) hh = 4;
  cv.fillRoundRect(cx - w / 2, cy - hh / 2, w, hh, 12, C_EYE);
  if (happy) cv.fillCircle(cx, cy + h / 2 + 6, w * 0.62, C_BG);   // masks lower half -> ^ shape
}
void drawEyes() {
  cv.fillScreen(C_BG);
  unsigned long now = millis();
  lookX += (lookTarget - lookX) * 0.25f;                  // smooth gaze
  int w = 46, h = 56, lid = 0, dx = (int)(lookX * 14);
  bool happy = false;
  if (mood == M_SHOCK) { w = 54; h = 78; }
  if (mood == M_HAPPY) happy = true;
  if (mood == M_CURIOUS) { h = 62; }
  if (blinkStart) {                                       // 220 ms blink
```

```
    unsigned long t = now - blinkStart;
    if (t > 220) { blinkStart = 0; nextBlink = now + random(1800, 5000); }
    else lid = (t < 110) ? t * 100 / 110 : (220 - t) * 100 / 110;
  } else if (now > nextBlink) blinkStart = now;
  drawEye(46 + dx, 64, w, h, lid, happy);
  drawEye(114 + dx, 64, w, h, lid, happy);
  if (mood == M_SHOCK) { cv.fillCircle(46 + dx, 64, 7, C_BG); cv.fillCircle(114 + dx, 64, 7, C_BG); }
  else if (!happy && lid < 60) { cv.fillCircle(46 + dx + dx / 3, 64, 9, C_BG); cv.fillCircle(114 + dx + dx / 3, 64, 9, C_BG); }
  tft.drawRGBBitmap(0, 0, cv.getBuffer(), 160, 128);
}

// ---------------- SENSORS ----------------
int distanceMM() {
#if USE_VL53L1X
  if (!tof.dataReady()) return -2;
  int d = tof.read(false);
  return tof.timeoutOccurred() ? -1 : d;
#else
  VL53L0X_RangingMeasurementData_t m;
  tof.rangingTest(&m, false);
  return (m.RangeStatus != 4) ? m.RangeMilliMeter : -1;
#endif
}
int lastDist = 9999;
unsigned long lastToF = 0, lastAlarm = 0, lastClap = 0;
bool obstacleLatch = false;

void checkToF() {
  if (millis() - lastToF < 60) return;
  lastToF = millis();
  int d = distanceMM();
  if (d == -2) return;
  if (d > 0 && d < 8000) lastDist = d; else if (d == -1) lastDist = 9999;
  if (lastDist < STOP_MM) {
    if (driveDir == 'F') stopMotors();
    setMood(M_SHOCK, 1200);
    if (millis() - lastAlarm > 2500) { sound(SND_ALARM); lastAlarm = millis(); }
    obstacleLatch = true;
  } else obstacleLatch = false;
}
void checkClap() {                          // peak-to-peak over ~20 ms window
  if (millis() - lastClap < 2000) return;
  int mn = 4095, mx = 0; unsigned long t0 = millis();
  while (millis() - t0 < 20) { int v = analogRead(PIN_MIC); if (v < mn) mn = v; if (v > mx) mx = v; }
  if (mx - mn > CLAP_LEVEL) {
    lastClap = millis(); sound(SND_CLAP);
```

```
    setMood(M_CURIOUS, 2500);
    lookTarget = (random(2) ? 1 : -1);
  }
}

bool solarOn = false; unsigned long lastSolarChk = 0;
void checkSolar() {                       // chime + happy eyes when sunlight starts charging
  if (millis() - lastSolarChk < 500) return;
  lastSolarChk = millis();
  bool on = analogRead(PIN_SOLAR) > 2000;  // ~>2.6V at pin = panel lit
  if (on && !solarOn) { sound(SND_CHARGE); setMood(M_HAPPY, 2500); }
  solarOn = on;
}

// ---------------- SIM7600E-H ----------------
String simBuf;
bool smsNext = false;

String simCmd(const char *c, unsigned long to = 1000) {
  SIM.println(c);
  String r; unsigned long t = millis();
  while (millis() - t < to) while (SIM.available()) r += (char)SIM.read();
  Serial.print("> "); Serial.println(c); Serial.println(r);
  return r;
}
void simInit() {
  SIM.begin(115200, SERIAL_8N1, SIM_RX, SIM_TX);
  delay(500);
  for (int i = 0; i < 5 && simCmd("AT", 500).indexOf("OK") < 0; i++) delay(500);
  simCmd("ATE0");
  simCmd("AT+CPIN?");
  simCmd("AT+CREG?");
  simCmd("AT+CLIP=1");              // caller ID
  simCmd("AT+CSDVC=1");             // audio path 1 = analog handset (MIC+/EAR+)
  simCmd("AT+CMGF=1");              // SMS text mode
  simCmd("AT+CNMI=2,2,0,0,0");      // deliver SMS directly as +CMT
}
void handleCommand(String c) {      // from SMS body: F B L R S H (happy)
  c.trim(); c.toUpperCase();
  if (!c.length()) return;
  lastDriveCmd = millis();
  switch (c[0]) {
    case 'F': if (lastDist >= STOP_MM) forward(); break;
    case 'B': backward(); break;
    case 'L': turnLeft(); break;
    case 'R': turnRight(); break;
```

```
    case 'S': stopMotors(); break;
    case 'H': setMood(M_HAPPY, 3000); sound(SND_HAPPY); break;
  }
}
void pollSIM() {
  while (SIM.available()) {
    char ch = SIM.read();
    if (ch != '\n') { if (ch != '\r') simBuf += ch; continue; }
    String line = simBuf; simBuf = "";
    if (!line.length()) continue;
    Serial.println("SIM: " + line);
    if (smsNext) { smsNext = false; handleCommand(line); }
    else if (line.startsWith("+CMT:")) smsNext = true;
    else if (line.startsWith("RING")) { SIM.println("ATA"); setMood(M_HAPPY, 4000); sound(SND_HAPPY); }
    else if (line.startsWith("NO CARRIER")) setMood(M_NORMAL, 0);
  }
}

// ---------------- SETUP / LOOP ----------------
void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 256; i++) sineTab[i] = 128 + (int)(70 * sin(i * 2 * PI / 256));   // volume = 70
  sndQ = xQueueCreate(6, sizeof(Snd));
  xTaskCreatePinnedToCore(audioTask, "audio", 3072, NULL, 1, NULL, 0);
  dacWrite(PIN_DAC, 128);
  motorsInit(); stopMotors();
  tft.initR(INITR_BLACKTAB); tft.setRotation(1); tft.fillScreen(C_BG);   // try INITR_GREENTAB if colours/offset wrong
  analogReadResolution(12); analogSetPinAttenuation(PIN_MIC, ADC_11db); analogSetPinAttenuation(PIN_SOLAR, ADC_11db);
  Wire.begin(21, 22);
#if USE_VL53L1X
  tof.setTimeout(500);
  if (!tof.init()) Serial.println("VL53L1X FAIL");
  else {
    tof.setDistanceMode(VL53L1X::Short);
    tof.setMeasurementTimingBudget(50000);
    tof.startContinuous(60);
  }
#else
  if (!tof.begin()) Serial.println("VL53L0X FAIL");
#endif
  sound(SND_STARTUP); setMood(M_HAPPY, 2500);
  nextBlink = millis() + 2000;
  simInit();
}
void loop() {
  unsigned long now = millis();
```

```
  if (mood != M_NORMAL && now > moodUntil) mood = M_NORMAL;
  if (mood == M_NORMAL && now > nextGlance) {          // idle: look around
    lookTarget = random(-10, 11) / 10.0f; nextGlance = now + random(1500, 4000);
  }
  checkToF();
  checkClap();
  checkSolar();
  pollSIM();
  if (driving && now - lastDriveCmd > CMD_TIMEOUT_MS) stopMotors();   // dead-man timeout
  static unsigned long lastFrame = 0;
  if (now - lastFrame >= 40) { lastFrame = now; drawEyes(); }
}
```

How to control the robot over 4G

- Send an SMS to the robot's SIM number containing a single letter: **F** forward, **B** backward, **L** left, **R** right, **S** stop, **H** happy eyes + sound. Each movement SMS drives for 1.5 s then stops (dead-man timeout): send again to continue.
- **Call** the robot number: it rings once, auto-answers, shows happy eyes and you have a two-way intercom through the MAX9814 mic (MIC+) and the speaker (EAR+ via the mixer).
- Forward (F) is refused when the laser sees an obstacle closer than 150 mm.

Step 5.3 — Full Code #2: ESP32-CAM live MJPEG stream

Board: 'AI Thinker ESP32-CAM'. **CPU frequency:** 240 MHz. **Flash frequency:** 80 MHz. Change `WIFI_SSID` and `WIFI_PASS` to your phone hotspot or home router (2.4 GHz only).

CloudRover_CAM.ino (Dedicated WiFi Video Streaming Server)

```
/* CloudRover V1.0 - ESP32-CAM (AI-Thinker, OV2640) live MJPEG stream
 * Board: "AI Thinker ESP32-CAM" | Partition: Huge APP | Upload with ESP32-CAM-MB shield
 * Open  http://<IP shown in Serial Monitor>/        (page with video)
 *       http://<IP>:81/stream                        (raw MJPEG stream)
 */
#include "esp_camera.h"
#include <WiFi.h>
#include "esp_http_server.h"

const char *WIFI_SSID = "YOUR_WIFI_NAME";      // <-- EDIT
const char *WIFI_PASS = "YOUR_WIFI_PASSWORD";  // <-- EDIT

// AI-Thinker pin map
#define PWDN_GPIO_NUM 32
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22
#define FLASH_LED 4

#define PART_BOUNDARY "123456789000000000000987654321"
static const char *STREAM_CT = "multipart/x-mixed-replace;boundary=" PART_BOUNDARY;
static const char *STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
static const char *STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";

httpd_handle_t web_httpd = NULL, stream_httpd = NULL;

static esp_err_t index_handler(httpd_req_t *req) {
  const char *html = "<html><body style='background:#111;color:#0ff;text-align:center;font-family:sans-serif'>"
                     "<h2>CloudRover Live</h2><img id='v' style='width:96%;max-width:640px;border:2px solid #0ff'>"
                     "<script>document.getElementById('v').src='http://'+location.hostname+':81/stream';</script>"
                     "</body></html>";
  httpd_resp_set_type(req, "text/html");
  return httpd_resp_send(req, html, strlen(html));
}
```

```
static esp_err_t stream_handler(httpd_req_t *req) {
  esp_err_t res = httpd_resp_set_type(req, STREAM_CT);
  if (res != ESP_OK) return res;
  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  char part[64];
  while (true) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (!fb) { res = ESP_FAIL; break; }
    res = httpd_resp_send_chunk(req, STREAM_BOUNDARY, strlen(STREAM_BOUNDARY));
    if (res == ESP_OK) {
      size_t n = snprintf(part, sizeof(part), STREAM_PART, fb->len);
      res = httpd_resp_send_chunk(req, part, n);
    }
    if (res == ESP_OK) res = httpd_resp_send_chunk(req, (const char *)fb->buf, fb->len);
    esp_camera_fb_return(fb);
    if (res != ESP_OK) break;                 // client disconnected
  }
  return res;
}

void startServers() {
  httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
  httpd_uri_t idx = {"/", HTTP_GET, index_handler, NULL};
  httpd_uri_t str = {"/stream", HTTP_GET, stream_handler, NULL};
  cfg.server_port = 80;
  if (httpd_start(&web_httpd, &cfg) == ESP_OK) httpd_register_uri_handler(web_httpd, &idx);
  cfg.server_port = 81; cfg.ctrl_port += 1;
  if (httpd_start(&stream_httpd, &cfg) == ESP_OK) httpd_register_uri_handler(stream_httpd, &str);
}

void setup() {
  Serial.begin(115200);
  pinMode(FLASH_LED, OUTPUT); digitalWrite(FLASH_LED, LOW);
  camera_config_t c;
  c.ledc_channel = LEDC_CHANNEL_0; c.ledc_timer = LEDC_TIMER_0;
  c.pin_d0 = Y2_GPIO_NUM; c.pin_d1 = Y3_GPIO_NUM; c.pin_d2 = Y4_GPIO_NUM; c.pin_d3 = Y5_GPIO_NUM;
  c.pin_d4 = Y6_GPIO_NUM; c.pin_d5 = Y7_GPIO_NUM; c.pin_d6 = Y8_GPIO_NUM; c.pin_d7 = Y9_GPIO_NUM;
  c.pin_xclk = XCLK_GPIO_NUM; c.pin_pclk = PCLK_GPIO_NUM; c.pin_vsync = VSYNC_GPIO_NUM;
  c.pin_href = HREF_GPIO_NUM; c.pin_sccb_sda = SIOD_GPIO_NUM; c.pin_sccb_scl = SIOC_GPIO_NUM;
  c.pin_pwdn = PWDN_GPIO_NUM; c.pin_reset = RESET_GPIO_NUM;
  c.xclk_freq_hz = 20000000; c.pixel_format = PIXFORMAT_JPEG;
  c.frame_size = FRAMESIZE_VGA;               // 640x480; use FRAMESIZE_QVGA if slow
  c.jpeg_quality = 12; c.fb_count = 2;
  c.fb_location = CAMERA_FB_IN_PSRAM; c.grab_mode = CAMERA_GRAB_LATEST;
  if (esp_camera_init(&c) != ESP_OK) { Serial.println("Camera init FAILED - check ribbon cable"); delay(3000); ESP.restart(); }
```

```
  sensor_t *s = esp_camera_sensor_get();
  s->set_vflip(s, 0); s->set_hmirror(s, 0);   // set 1 if image is upside down / mirrored

  WiFi.mode(WIFI_STA); WiFi.setSleep(false);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  unsigned long t = millis();
  while (WiFi.status() != WL_CONNECTED) {
    delay(300); Serial.print(".");
    if (millis() - t > 20000) { Serial.println("\nWiFi failed, restarting"); ESP.restart(); }
  }
  startServers();
  Serial.print("\nCloudRover camera ready: http://"); Serial.println(WiFi.localIP());
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) { WiFi.reconnect(); delay(5000); }   // keep link alive
  delay(1000);
}
```

Flash procedure with the ESP32-CAM-MB shield

- [ ] Seat the ESP32-CAM board into the MB shield (USB socket faces outward, camera is on top).
- [ ] Plug the micro-USB into the PC.
- [ ] Select board: **AI Thinker ESP32-CAM** and your COM port.
- [ ] Hit **Upload**. The MB shield has auto-download transistors and drops GPIO 0 to GND for you. (If upload fails with 'Timed out waiting for packet header', hold the IO0 button on the shield, press RST once, then release IO0 and retry.)
- [ ] Open Serial Monitor at **115200**. Press the RST button on the shield: it prints the assigned IP address (e.g. `http://192.168.43.120`). Write this down.
- [ ] Open that IP in a browser on the same WiFi network (phone or laptop). You will see the live video stream.
- [ ] Unplug from the PC, remove the MB shield, and mount the bare ESP32-CAM on the faceplate.

Watching the video away from home

The ESP32-CAM does not route through the SIM7600 (it has no modem connection). For wide-area use: (1) set your phone as a mobile hotspot with the same SSID and password the camera is compiled for; (2) turn the hotspot on: the camera connects to your phone and you view it on your phone's browser; or (3) use a router port-forward / ngrok-style tunnel if the robot is at home.

PHASE 6 — Final Physical Assembly & Stacking



Figure 6.1 — Diagram 5: 4-Tier Mechanical Stacking & Hardware Elevation.

Assembly order

Step 1 — Base: motors, tracks and batteries

- [ ] Screw the two geared motors to the tank chassis plate. Pass their wires through the plate's motor slots.
- [ ] Bolt Part 1 (Base Tray) down onto the chassis plate using 4 x M3 screws through the floor holes.
- [ ] Mount the ball caster to the rear underside of the base tray using 2 x M3 screws and nuts.
- [ ] Screw the 2-slot 18650 holder into the rear pocket of Part 1 using two M2.5 or M3 screws. Pass the holder's red/black wires through the floor window into the tray.
- [ ] Mount the L298N module on its 4 posts in the front half of Part 1 with 4 x self-tap M2.5 x 6 screws. Connect the left motor to OUT1/OUT2 and right motor to OUT3/OUT4.

Step 2 — Layer 1 (power)

- [ ] Screw 4 x **M3 x 6 mm brass standoffs** into the four Layer 1 boss holes in the floor of Part 2 (Cubic Shell).
- [ ] Seat the 5 x 7 cm perfboard (Layer 1: power rails, capacitors, PAM8403 and the regulator) on these 4 standoffs.
- [ ] Fasten the board to the standoffs with four **M3 x 25 mm standoffs** (they screw straight into the 6 mm standoffs below, clamping the board and creating the posts for Layer 2).
- [ ] Pass the power and motor wires from the base tray up through the floor window: connect the battery holder to TP4056 B+/B-, regulator output to 5 V rail, and motor leads to the L298N.

Step 3 — Layer 2 (ESP32 / SIM7600 / audio)

- [ ] Prepare the 70 x 65 mm perfboard with 4 x dia 3.2 corner holes matching the 4 x 25 mm standoffs (62 x 57 mm pattern).
- [ ] Mount the ESP32 NodeMCU and SIM7600E-H side-by-side using female headers or standoffs.
- [ ] Solder the passive mixer components (10 uF cap + 2 x 10 k resistors) in the spare corner of Layer 2.
- [ ] Drop Layer 2 onto the four 25 mm standoffs and screw down with 4 x M3 x 6 screws.

- [ ] Plug in the Dupont wires from Layer 1 (5 V rail, GND rail, PAM8403 L_IN) and from the L298N (IN1..IN4, ENA, ENB) according to the Phase 4 table.

Step 4 — Rear column: SIM slot, single Type-C and rocker switch

- [ ] Slide the SIM into the SIM7600 module's tray (or plug the flat extension into it if your module faces away from the rear wall).
- [ ] Screw the TP4056 board to the two rear wall bosses so its USB Type-C port protrudes flush into the 9.8 x 3.8 mm cut-out. Test-fit a USB cable: the plug must click home fully.
- [ ] Push the mini rocker switch into its 15.2 x 10.2 mm cut-out until the plastic clips snap out.
- [ ] Peel the backing off the 4G FPC stick-on antenna and adhere it flat against the **right inner wall** (away from the ESP32-CAM and motors). Plug the IPEX connector into the SIM7600 MAIN antenna socket until it clicks.
- [ ] Glue the 8 ohm 1 W speaker into its ring on the left inner wall (behind the grille) with a ring of silicone or hot glue. Connect its two wires to PAM8403 L_OUT+ and L_OUT-.

Step 5 — Front faceplate, then solar lid

- [ ] Mount the ST7735 TFT into the faceplate window using 4 x M2 x 4 screws into the printed posts. A foam tape bezel prevents light leaks.
- [ ] Seat the MAX9814 capsule into the 1.6 mm pinhole recess with a dab of silicone.
- [ ] Seat the VL53L0X/L1X behind the 10 x 6 mm window. Screw with two small self-tap screws.
- [ ] Mount the bare ESP32-CAM board behind the 9 mm lens hole. Plug its 5 V and GND wires to the 5 V rail (closest to the 1000 uF cap).
- [ ] Plug all remaining Dupont wires from the faceplate to the ESP32 pins on Layer 2 (see Phase 4 table).
- [ ] Tuck the wiring neatly into the cube using Kapton tape loops. Keep the audio wires away from the motor wires.
- [ ] Fasten the faceplate (Part 3) with 4 x M2.5 x 8 self-tapping screws at the four corners.
- [ ] Feed the solar panel leads through the 3 mm hole in the top lid (Part 4). Press the panel into the 2 mm pocket with a thin bead of silicone along its edges. Solder the panel red wire to the 1N5819 diode and black wire to TP4056 IN- (Step 3.1).
- [ ] Fasten the top lid (Part 4) with 4 x M2.5 x 8 countersunk screws.

PHASE 7 — First Boot Testing, Troubleshooting & Defense Guide

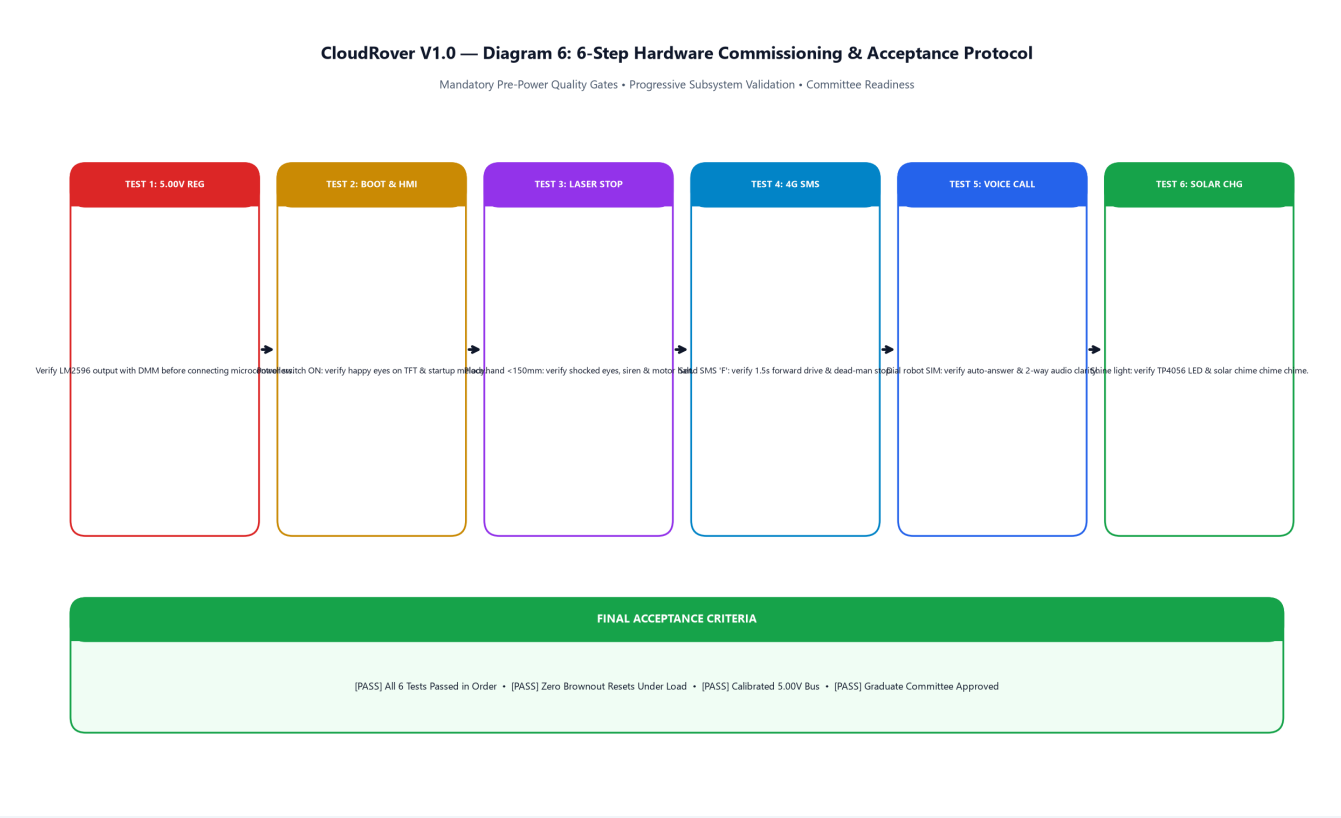


Figure 7.1 — Diagram 6: 6-Step Hardware Commissioning & Acceptance Protocol.

7.1 The 6-step first-power-on test protocol

[ ] **TEST 1 — Voltage verification under zero load.** Open the lid. Switch the rocker ON. Put multimeter probes on the 5 V perfboard rail. It must read **5.00 V (+/- 0.05 V)**. If below 4.8 V or above 5.1 V, adjust the trimmer immediately.

[ ] **TEST 2 — Eyes and sound boot test.** Watch the 1.8-inch TFT. Within 1.5 seconds it must display cyan robot eyes, play the three-note ascending startup melody from the speaker, and start the slow blink loop (blinks every 3-5 seconds). If the screen stays black, check the 3.3 V rail and SPI wires (GPIO 5, 4, 2, 23, 18).

[ ] **TEST 3 — Laser emergency stop test.** Put your hand 30 cm in front of the robot: the eyes look forward. Move your hand to **within 150 mm (15 cm)** of the front faceplate: the eyes must instantly widen to 'shocked', the alarm siren plays and if the robot was driving forward it halts immediately. Remove your hand: after 2.5 seconds it returns to normal.

[ ] **TEST 4 — Clap / sound interaction test.** Clap your hands loudly 30 cm from the robot. The eyes become round ('happy') and it plays the curious high chirp. If it does not respond, adjust the threshold in firmware or tie the MAX9814 GAIN pin to GND.

[ ] **TEST 5 — 4G cellular and voice test.** (1) Insert the SIM and power on. Open Serial Monitor: wait for SMS ready (typically 15-30 seconds). (2) Send an SMS with the single letter **F**: the robot drives forward for 1.5 seconds and stops. Send **H**: it shows happy eyes and plays the chirp. (3) Call the robot number from your phone: it auto-answers on the first ring, shows happy eyes, and you can speak and hear through the intercom.

[ ] **TEST 6 — Solar charging LED test with a flashlight.** Shine a phone flashlight (or sunlight) straight onto the solar lid at 5 cm. The TP4056 CHRG LED lights red (if the batteries are not full), the chime plays and the eyes become happy (GPIO 35 sense above about 2.6 V). Cover the panel: the LED goes out. Plug USB-C: the LED stays lit, proving both sources work through one port.

7.2 Troubleshooting table

Symptom	Likely cause	Fix
ESP32 brownout reset ('Brownout detector was triggered'), reboot loop when modem registers or motors start	5 V rail sags (weak cells, thin/long wire, boost limited, no caps)	Check the caps are on the rail, use 0.5 mm wire, charge the cells, lower DRIVE_SPEED to 150, power the L298N VS from battery [A], add 470 uF at ESP32 VIN
SIM7600 does not answer 'AT'	UART TX/RX not crossed, wrong baud, module not powered up	Swap GPIO 16/17 wires; press module PWR key for 1 s; try 9600 or 115200 (code uses 115200)
CREG shows 0,2 / 0,3 / 0,0 (not registered)	No antenna, band mismatch, SIM PIN, weak signal	Attach antenna first; SIM from carrier supported by E-H bands; remove PIN; move near window; wait 60 s
No auto-answer	No RING seen / CLIP off	Check 'AT+CLIP=1' returned OK; 4G voice needs VoLTE or CSFB: if carrier is LTE-only, module may not receive voice (use 2G/3G-fallback SIM)
No voice on the call	Mic path wrong / audio path	Confirm MIC+ has 1 uF capacitor and MIC-/EAR- go to GND; try AT+CSDVC=3 or AT+CMICGAIN=4 / AT+CLVL=5
One or both motors run backwards	Motor wires polarity	Swap the two wires of that motor at L298N OUT terminals (do not change the code)
Robot turns wrong way (L/R inverted)	Left/right motors swapped	Swap left motor to OUT3/OUT4 and right to OUT1/OUT2, or exchange IN1<->IN3 and IN2<->IN4 wires
Motors do nothing	ENA/ENB jumpers still fitted or PWM too low; VS unpowered	Remove jumpers; PWM 200+ in code; check 5 V on VS and rail
Audio hiss / motor whine in speaker	Shared ground noise, long wires	Star-ground to rail, twist speaker wires, add 100 nF across each motor terminal, add 220 uF at PAM8403, lower amplitude
Clicks every time screen refreshes	SPI noise in audio path	Keep audio wires away from SPI; add 1 uF coupling and move DAC wire to 10 k near PAM
TFT white or wrong colours	Wrong init tab or CS/DC swapped	Try INITR_GREENTAB or REDTAB; verify GPIO 5 (CS), GPIO 2 (DC), GPIO 4 (RST)
VL53L0X FAIL printed	I2C wiring or wrong chip	SDA 21, SCL 22, 3.3 V; use VL53L1X option if you bought an L1X
Upload error 'invalid header' / boot loop	GPIO 12 pulled high or board in bootloader mode	Disconnect L298N IN1 wire while uploading; hold BOOT while 'Connecting...'
ESP32-CAM 'Camera init failed 0x105'	Ribbon cable or power	Re-seat camera ribbon (black latch), use 5 V rail with 1000 uF
ESP32-CAM image upside down/mirrored	Mounting orientation	Set s->set_vflip(s,1) and/or set_hmirror(s,1) in code
Solar chime plays all the time	GPIO 35 divider floating / lamp lighting	Check the 2 x 100 k; raise threshold 2000 to 2600

7.3 Top 5 questions from the graduation committee — model answers

Q1. Why use both a 4G modem and WiFi? Why not 4G video?

Each link matches a need. The SIM7600E-H gives wide-area control and voice (SMS commands and a two-way call) with only a few kbit/s of data. Live video at a useful quality needs a continuous stream of 1-2 Mbit/s that the ESP32-CAM cannot route through the modem, so video is on WiFi (local, or via a phone hotspot). A 4G video link through the modem's data connection is the stated V2 improvement.

Q2. How did you make the power system safe and reliable?

Two matched 18650 cells with a protected charge board (DW01A over/under-voltage and short-circuit protection), a single charging port, a Schottky diode to isolate the solar input, a regulator calibrated to 5.00 V BEFORE connecting any microcontroller, 0.5 mm copper for high-current paths, and 1000 uF + 100 nF capacitors next to the 4G modem and the camera to absorb the 2 A transmit bursts. Boot-strap pins were checked and a star ground was used.

Q3. What does the solar panel actually contribute?

A 6 V panel of 1 W gives about 150-250 mA in direct sun, which is roughly 5-10 % of a 2 A peak load, but the robot idles at about 150-250 mA. So it can extend stand-by time and trickle-charge while parked. It is a supplement, not a replacement for the USB-C charge, and we measured and reported it rather than claiming self-sufficiency.

Q4. How does the robot avoid collisions and what happens when communication fails?

The VL53L0X/L1X laser ranges every 60 ms in the main loop, independent of the network. Below 150 mm the code stops the motors immediately, shows shocked eyes and plays an alarm. Every remote movement command also auto-expires after 1.5 s (dead-man timer), so a lost SMS or call can never leave the robot driving.

Q5. How do the audio and the eyes work without extra audio hardware or files?

The ESP32's built-in DAC (GPIO 25) generates sine tones from a lookup table inside a separate FreeRTOS task, so sound never blocks the animation. A passive mixer (two 10 kohm resistors and a 10 uF capacitor) adds those tones to the 4G modem's audio output into one PAM8403 amplifier. The MAX9814 microphone output is split: one branch to the modem for calls, one to an ADC pin for clap detection. A shared mood state machine ties each sound to an eye expression.

7.4 Five-minute defense presentation script

Time	Slide / action	What to say
0:00	Title + live robot on the table	'This is CloudRover: a 4G-connected, solar-assisted tracked robot with an animated face.' Switch on: startup melody and happy eyes.
0:40	Problem and use cases	'Most small robots stop at the WiFi range. CloudRover is controlled and heard through the mobile network, with inspection, security and telepresence uses.'
1:20	System block diagram (Diagram 2/3)	Explain the two-ESP32 split: one for control and the other for the camera; the power path through a single USB-C and a solar roof.
2:10	CAD blueprint (Diagram 1)	'80 x 80 x 85 mm cube in four printed parts; batteries in the base for a low centre of gravity.'
2:50	Live demo	Hand in front: shocked eyes and stop. Clap: curious chirp. Send SMS 'F'. Phone call: answer and speak. Flashlight: chime.
4:00	Results and limits	Report measured values: regulator 5.00 V, stop distance 150 mm, call latency, solar current. State the limits honestly (video is WiFi; solar is a supplement).
4:40	Future work + thanks	4G video via modem data, a map/GPS (SIM7600 has GNSS), an autonomous patrol mode, a custom PCB.

7.5 Final acceptance checklist

- [ ] Regulator measured 5.00 V and the trimmer marked.
- [ ] All 6 commissioning tests passed in order.
- [ ] Faceplate, rear column (SIM / ONE USB-C / switch) and lid fitted.
- [ ] Robot drives forward, back, left, right and stops when commanded.
- [ ] Photographs and measurements recorded for the final engineering report.
- [ ] Spare backups: the two .ino files, the STL files and this PDF backed up on a USB drive.

[VERIFIED ACCEPTANCE]

Congratulations: CloudRover V1.0 hardware commissioning is complete and verified for graduation defense!